AiFood — Compras pelo WhatsApp · Entrega única

Time: Davi, Fabio, Pedro e Breno · Hackathon SEMCOMP × iFood "Vendendo com Agentes" (ONOVOLAB, São Carlos), 17–18/10/2026. Produto: AiFood, compras pelo WhatsApp, assistente de compras do iFood que funciona dentro do WhatsApp (agente cadastrado no portal como "Compras pelo WhatsApp"; o projeto passou a se chamar AiFood às 10h26 de 18/10).

Como ler. Este arquivo junta os entregáveis 01 a 06 e dois anexos. A parte 06 (feedback sobre o MCP) e o Anexo A (cronologia) são as mais detalhadas, porque quem lê é o time que mantém o MCP. Toda afirmação sobre o MCP leva **[testado]** (vimos acontecer) ou **[só na doc]** (lemos na documentação e não reproduzimos). "Não registrado" quer dizer que o repositório não guarda aquele dado. Horários em Brasília; os do git são a referência quando há divergência.

Fontes: `docs/entregaveis/01` a `06`, `docs/contexto-evento.md`, `docs/decisoes.md`, `docs/estado.md`, `docs/qa-18-10.md`, `docs/como-funciona.md`, `docs/whatsapp.md`, `docs/referencias.md`, `docs/prompts/*.md`, `regras da api/ifood-mcp-developer-portal.json`, `.coord/claims/*.md`, o código em `src/` e o `git log` (264 commits até 18/10 10h26).

---

01 Proposta de Negócio

1.1 O problema

Pedir no app de delivery exige navegar por telas, filtros, carrinho e pagamento. Para quem tem pouca prática com aplicativos, cada tela é um ponto de desistência. Essas pessoas já usam o celular, muitas vezes só para conversar.

Dados (IBGE, PNAD Contínua TIC 2023):

- O uso de internet entre pessoas com 60 anos ou mais subiu de 24,7% (2016) para 66,0% (2023).
- Entre quem usa internet, 91,1% enviam ou recebem mensagens por aplicativos (texto, voz ou imagem).

Leitura do time (argumento, não dado medido): a conversa é a interface que esse público já domina; o app de delivery não é.

1.2 Para quem

- Pessoas idosas ou com pouca familiaridade digital, que conversam no WhatsApp mas evitam apps.
- Quem repete quase sempre a mesma compra (mercado da semana, lanche de sempre).
- Grupos que querem dividir um pedido (lanche com amigos).

1.3 Proposta de valor

Pedir do iFood como quem manda mensagem para alguém de confiança.

- **Fala do jeito que a pessoa quer:** texto, áudio ou foto (lista de papel, embalagem, foto de comida). Responde por escrito, ou por áudio quando a pessoa pede.
- **Nunca deixa a pessoa perdida:** toda mensagem termina dizendo o que fazer, com botões.
- **Repete a compra com 1 toque:** "toda semana às 18h". No dia, o lembrete mostra itens, endereço e valor e só pede com o "Sim, pode pedir" da pessoa.
- **Ajuda a fechar o pedido:** perto do pedido mínimo, sugere o que combina; para grupos, monta um kit numa loja só e diz quanto fica para cada um.
- **Segurança por desenho ("a IA entende, o código garante"):** a IA nunca confirma compra. O pedido sai por um destes caminhos, todos iniciados pela pessoa: toque em "Confirmar pedido" ou um "sim" dela no resumo; "já pode passar no pix", que fecha direto, sem resumo; ou "Sim, pode pedir" no lembrete da compra recorrente, que também fecha direto (mostra o resumo antes só se o preço subiu mais de 10%). O CPF nunca passa pela IA, e a pessoa pode apagar seus dados.

1.4 O que já existe: o Ailo

O iFood já tem o Ailo, agente próprio no WhatsApp (lançado em out/2025 e em beta, segundo a imprensa). É a prova de que o WhatsApp é um bom canal para o iFood. Nossa proposta é complementar e olha para o espaço ao lado:

- compra da semana (mercado, padaria, farmácia), com vários itens no mesmo pedido;
- quem conversa no WhatsApp mas não abre o app, com pedido por texto, áudio ou foto da lista de papel;
- compra que se repete e pedido em grupo com valor por pessoa.

Os dois usam o mesmo ecossistema (lojas, pagamento, entrega e rastreio do iFood).

1.5 Por que interessa ao iFood

Hipóteses do time, a validar com dados reais:

- **Novo canal para um público pouco atendido:** o pedido continua sendo do iFood; só muda a porta de entrada.

- **Mais frequência:** a recorrência transforma a compra habitual em hábito. O MCP não tem recorrência nativa; construímos uma por cima.
- **Ticket e conclusão:** sugestões para atingir o pedido mínimo e itens que combinam reduzem pedidos que travam.
- **Sem integração nova:** usa o MCP oficial de consumidor, com o login do próprio cliente.

1.6 Como medir

Propostas de métrica, **ainda não medidas**: pedidos concluídos sem ajuda humana sobre conversas iniciadas; mensagens até o pedido fechado; pessoas com recorrência ativa e quantas confirmam o lembrete; desistência por etapa (endereço, escolha, pagamento).

1.7 Viabilidade e limites

- Funciona de ponta a ponta no mock (login, endereço, busca, carrinho, resumo, confirmação, rastreio e lembrete), com testes automatizados e QA manual. No MCP real, o bot chegou a fazer e rastrear pedido (parte 06, D16).
- Limites do MCP que afetam o produto (detalhe na parte 06): bloqueio por rajada de requisições, falta de tool para listar pedidos e respostas pesadas para chat.
- Pagamento só PIX e vales: cartão exige um token que nenhuma tool fornece.

1.8 Próximos passos

Medir com pessoas do público-alvo; ampliar lojas e categorias.

Fontes: IBGE, PNAD Contínua TIC 2023 · Canaltech sobre o Ailo · Consumidor Moderno sobre o Ailo

---

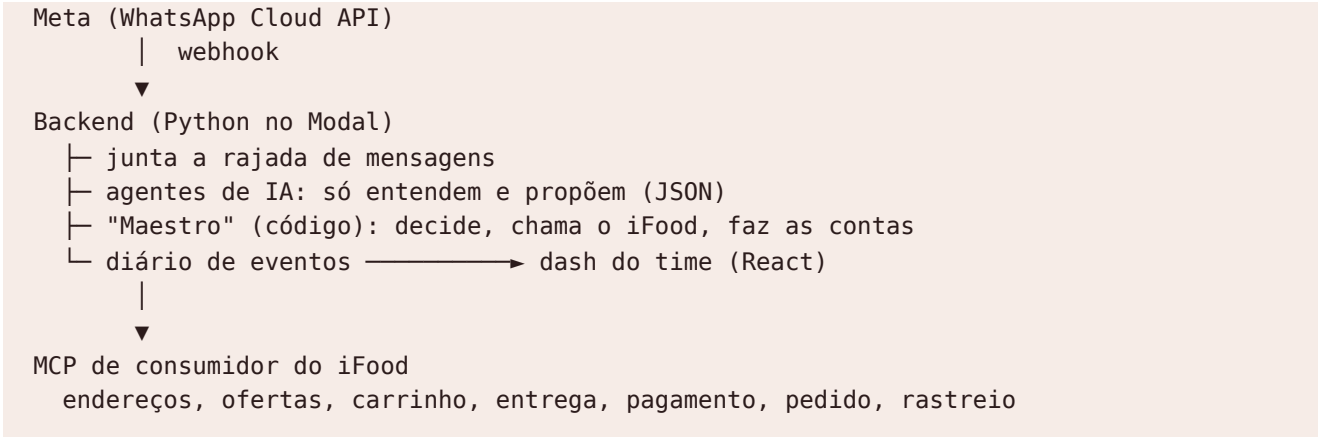
02 Desenho de Solução

2.1 Visão em uma frase

A pessoa conversa por texto, áudio ou foto; a IA entende o pedido e o código conduz cada passo no MCP do iFood até a confirmação da pessoa.

2.2 As peças

```
Pessoa no WhatsApp
      │  texto / áudio / foto / toque em botão
      ▼
```



2.3 Fluxo principal

1. **Primeiro contato.** "oi" ou o pedido direto. O bot cumprimenta e mostra uma vitrine de ideias.
2. **Login no iFood (uma vez).** Botão "Entrar no iFood" abre o login web oficial (OAuth). Na volta, o bot diz de qual conta é e retoma o que a pessoa tinha pedido.
3. **Endereço.** Vem da conta iFood: com 1, usa direto; com vários, confirma o provável; sem nenhum, pede e salva.
4. **Entender o pedido.** "pão, leite e café", um áudio ou a foto de uma lista viram itens com quantidade e tipo de loja.
5. **Escolher a loja.** Lista com 2+ itens vai para a loja que tem tudo e sai mais barata (itens × quantidade + entrega). O iFood fecha um pedido por loja.
6. **Mostrar opções.** Carrossel de cards com foto e "Quero esse" (até 10 por vez no texto, 3 no modo áudio), mais "Mais opções". A pessoa toca, digita o número ou diz "o do meio".
7. **Carrinho.** Opções obrigatórias uma de cada vez, depois a quantidade na unidade de venda (unidade, kg, pacote, dúzia). Abaixo do mínimo, o bot sugere o que combina.
8. **Fechar.** "🧾 Revisar pedido" ou "pode fechar": entrega padrão, PIX ou vale, melhor cupom, CPF da nota se não estiver na conta (pode pular, salvo loja que exige) e o resumo com itens, entrega, pagamento, endereço e total.
9. **Confirmação.** Toque em "Confirmar pedido" ou um "sim" da pessoa no resumo faz o pedido. "Já pode passar no pix" fecha direto, sem resumo e sem CPF na nota.
10. **Depois do pedido.** O bot acompanha o status no ritmo que o iFood indica e avisa cada mudança. "Cadê meu pedido?" e "qual o código de entrega?" são respondidos a qualquer momento.

2.4 Fluxos laterais

- **Compra recorrente.** Oferecida na 2ª compra do mesmo item ou categoria, ou quando a pessoa pede. No dia, o lembrete mostra itens, loja, endereço, pagamento e valor da última vez. "Sim, pode pedir" refaz o carrinho e fecha (resumo antes só se o preço subiu mais de 10%); "Hoje não" adia; "Parar lembretes" desliga. Sem o sim, nada é pedido.

- **Pedido em grupo (kit).** "somos 6, lanche até R$ 100" vira um kit numa loja só. A IA propõe a composição; o código respeita orçamento e restrições e faz a conta por pessoa. Botões "Quero esse kit", "Outra loja", "Não quero"; o carrinho só é criado no "Quero esse kit".
- **Foto.** Comida: pergunta se procura algo parecido. Lista ou produto: vira a lista do pedido. Outra coisa: comenta e volta ao ponto. Na etapa do CPF a foto nem é aberta.
- **Quem some ou muda de ideia.** Cutucada após 1 min de silêncio numa pergunta; carrinho local esquecido após 5 min; após 30 min, volta ao menu. "Cancelar", "mudar" ou um pedido novo são entendidos em qualquer etapa.
- **Quando o iFood falha.** Freio de chamadas, disjuntor por container, ponte por outro IP e, no modo de apresentação, troca para um catálogo de demonstração (detalhe na parte 06, D6).

2.5 Quem decide o quê

Etapa	IA (propõe)	Código (garante)
Entender a mensagem	intenção, itens, quantidade	regras rápidas antes da IA; plano B sem IA
Ler a foto	tipo da foto, itens da lista	documento e CPF nunca vão para a IA
Escolher a loja	—	soma itens + entrega, escolhe a mais barata com tudo
Montar o kit	composição	orçamento, restrições, conta por pessoa
Fechar o pedido	—	entrega, pagamento, cupom, resumo
Confirmar	nunca	só a pessoa (toque, "sim", "já pode passar no pix", "Sim, pode pedir")

2.6 Fora do escopo

Cartão de crédito, retirada na loja, entrega agendada, cancelamento e reembolso pela API, divisão do pagamento entre amigos (o bot só informa quanto fica para cada um).

---

03 Racional da Experiência

Princípio: pedir do iFood deve parecer mandar mensagem para alguém de confiança. Cada decisão responde a "onde uma pessoa com pouca prática desistiria?". Todas foram testadas à mão no WhatsApp, em rodadas de QA de 18/10 entre 01h e 05h30, e várias mudaram por causa do teste (`docs/qa-18-10.md`).

#	Decisão	Por quê	O que o teste mudou
1	Aceitar texto, áudio e foto; responder por áudio quando a pessoa pede	quem não domina o app domina a conversa	áudio com música de fundo funcionou; lista de papel por foto virou carrinho. Quantidade segue a unidade de venda ("12 ovos" numa dúzia = 1 dúzia)
2	Várias mensagens curtas valem como uma	quem tem pouca prática manda frases picadas	espera adaptativa: toque ~0,6 s, "oi" curto ~4 s, texto ~2,5 s, áudio ~1,8 s, teto 9 s; trava contra resposta duplicada
3	Toda resposta termina com o próximo passo, só com botões que levam a algo	o pior momento é não saber o que fazer	"Mudar" digitado no pagamento não funcionava e não havia volta no pagamento e no CPF: corrigidos
4	Opções com foto em carrossel, "Mais opções" reaproveita a lista já buscada	foto reduz erro de escolha	o carrossel mostra até 10 cards por vez; no modo áudio, 3 ditos em voz
5	O bot escolhe o que dá: entrega padrão, melhor cupom, loja que tem tudo	cada pergunta a menos é uma desistência a menos	o "até agora" passou a separar itens de "com entrega e taxas" (M2)
6	Só a pessoa fecha o pedido; a IA nunca	pedido errado custa dinheiro e confiança	um "sim" ao lado de uma sugestão fechava por engano; a sugestão passou a valer uma mensagem só, e o "sim" é sempre para o pedido
7	Recorrência oferecida na 2ª compra ou quando pedida	maior economia de esforço para quem repete	"Não precisa" pausa a oferta uma semana; "Hoje não" diz quando volta; aviso falso de "preço mudou" corrigido
8	Kit em grupo numa loja só com valor por pessoa	a conta é o que mais gera discussão	foi o ponto mais frágil: ajustes refaziam o kit do zero. Hoje o ajuste é aplicado pelo código sobre o kit da tela
9	Segurança visível	confiança do público	CPF da conta iFood ou pedido uma vez (pode pular, salvo loja que exige), nunca vai a modelo; aviso de maioridade para álcool; papo fora do assunto recusado (política da Meta contra chatbot genérico); "sair" e "apagar meus dados"
10	Falha do iFood não aparece para a pessoa	o bot não pode travar quando ela mais precisa	botão antigo gera um aviso só; troca para o catálogo de reserva leva o carrinho junto, pelo nome

Como validamos: QA manual com cerca de 35 itens registrados (ids B, U, M, C, O, H), quase todos corrigidos até ~06h45 de 18/10; 545 testes automatizados no back (pytest) e 54 no dash (Vitest), rodados para este documento. **Não testamos com pessoas do público-alvo**; é o primeiro passo depois do hackathon.

---

04 Interface Visual

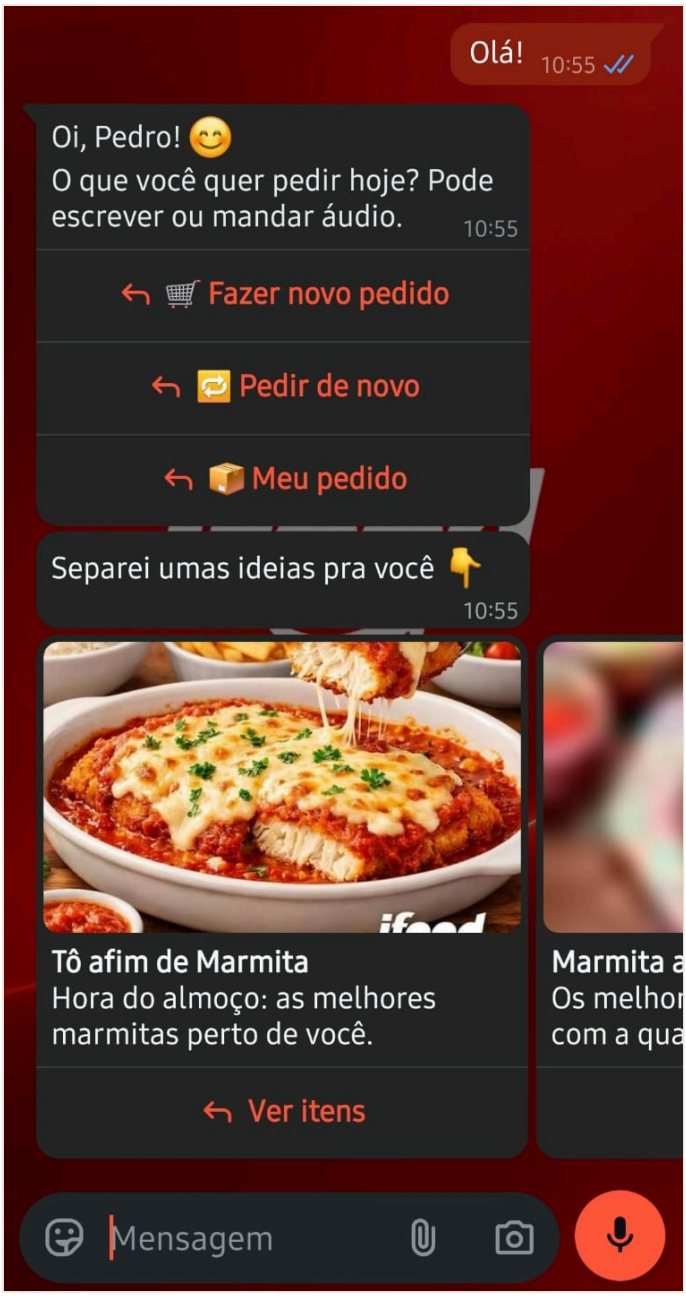
O produto tem duas interfaces: **a conversa no WhatsApp**, para quem compra (nenhum app novo: botões, listas e cards com foto), e **o dash do time**, para quem opera (React +

Vite).

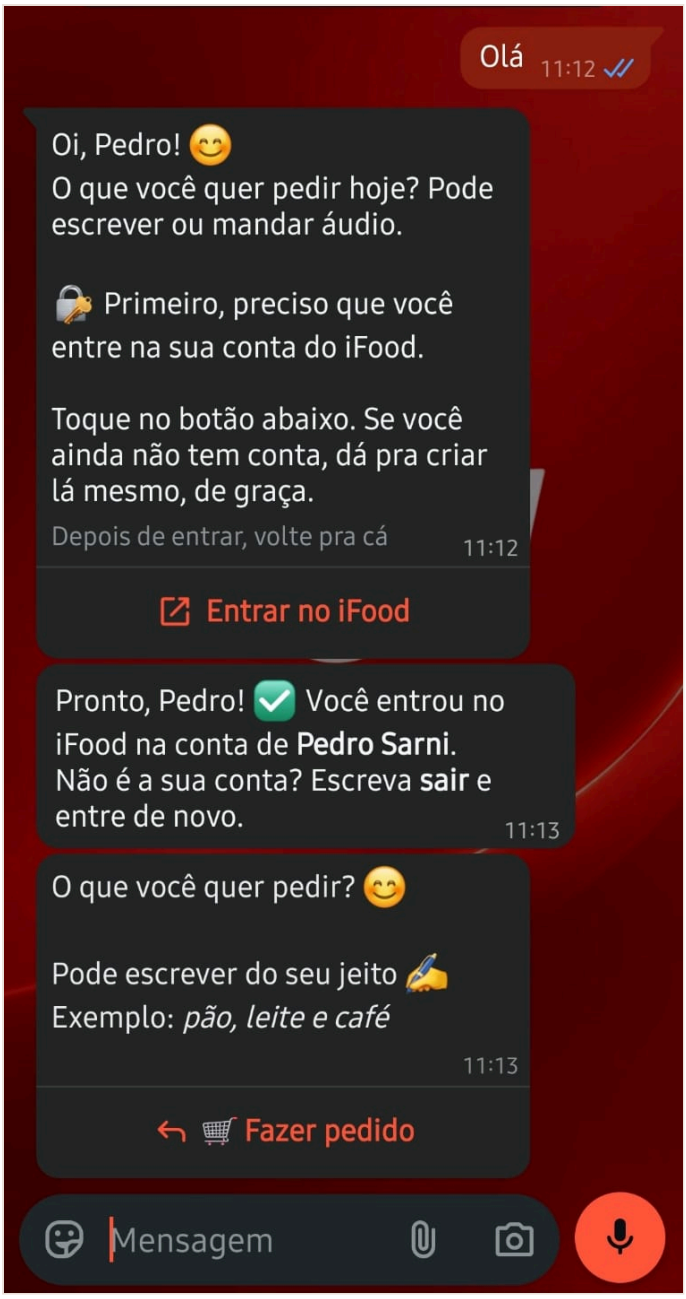
Parte A — A conversa no WhatsApp

Regras visuais: frases curtas, um emoji por ideia (🛒 pedido, 🏠 endereço, 💳 pagamento, 💰 valor), negrito no que importa, toda mensagem termina com o próximo passo e nada em inglês chega à tela.

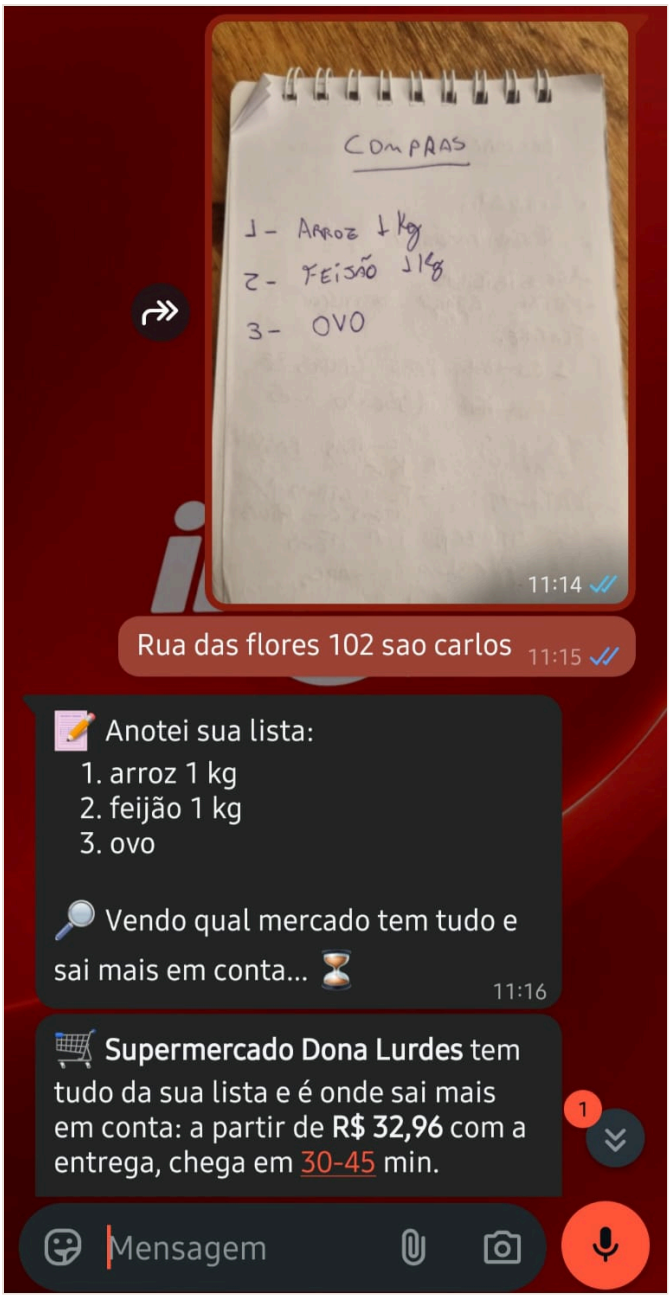
A1. Boas-vindas. "Oi, Maria! 😊 O que você quer pedir hoje? Pode escrever ou mandar áudio." Logo abaixo, a vitrine "Separei umas ideias pra você 👇".



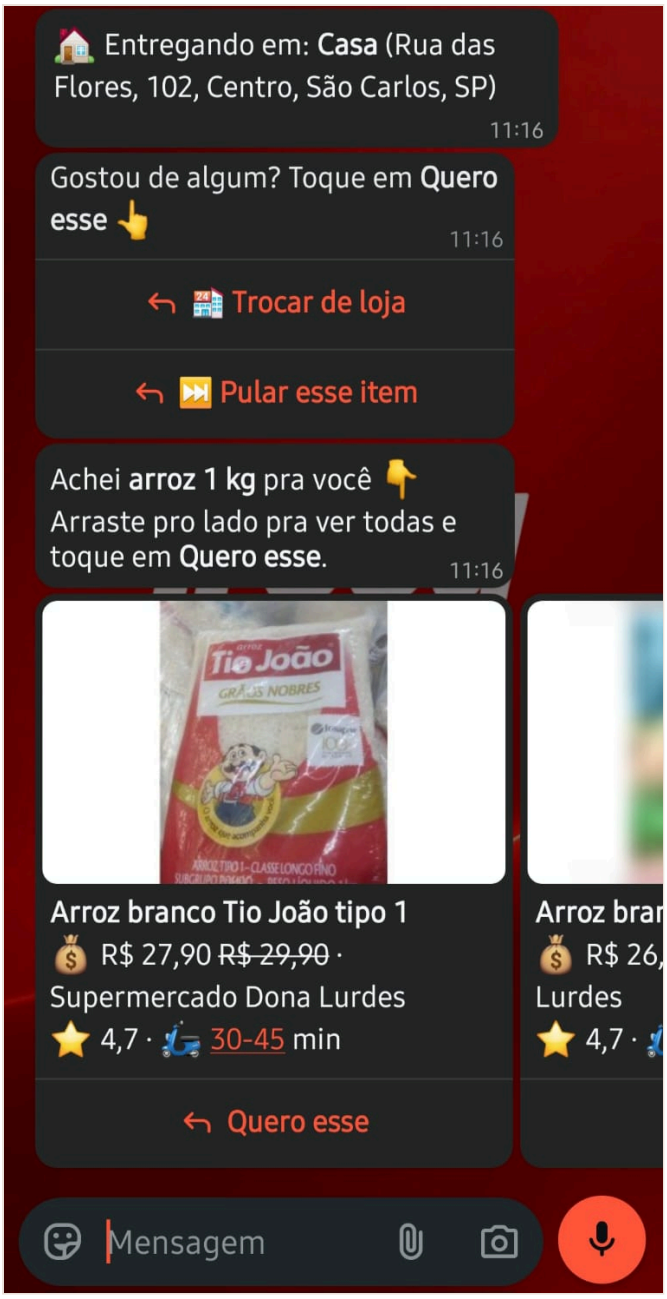
A2. Login no iFood. "🔐 Primeiro, preciso que você entre na sua conta do iFood." Botão **Entrar no iFood**. Na volta, o bot diz de quem é a conta ("Não é a sua conta? Escreva *sair*").



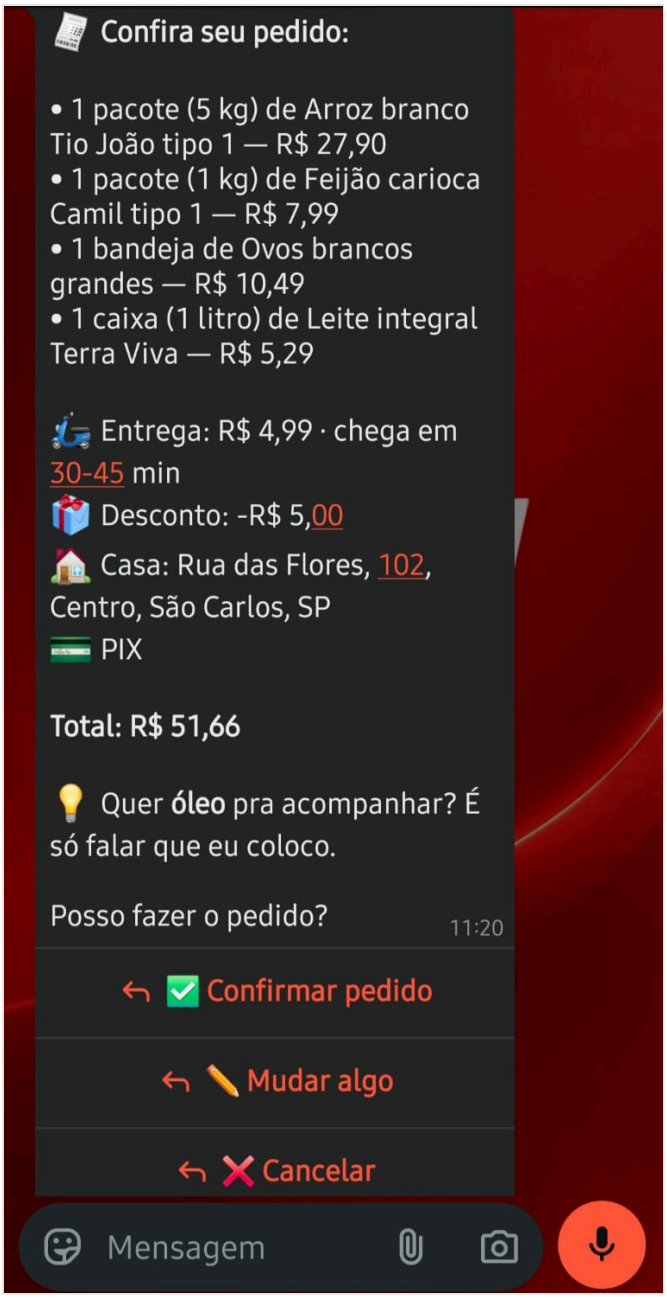
A3. Lista entendida. "📝 Anotei sua lista: 1. 2x leite · 2. 1x pão · 3. 1x café".



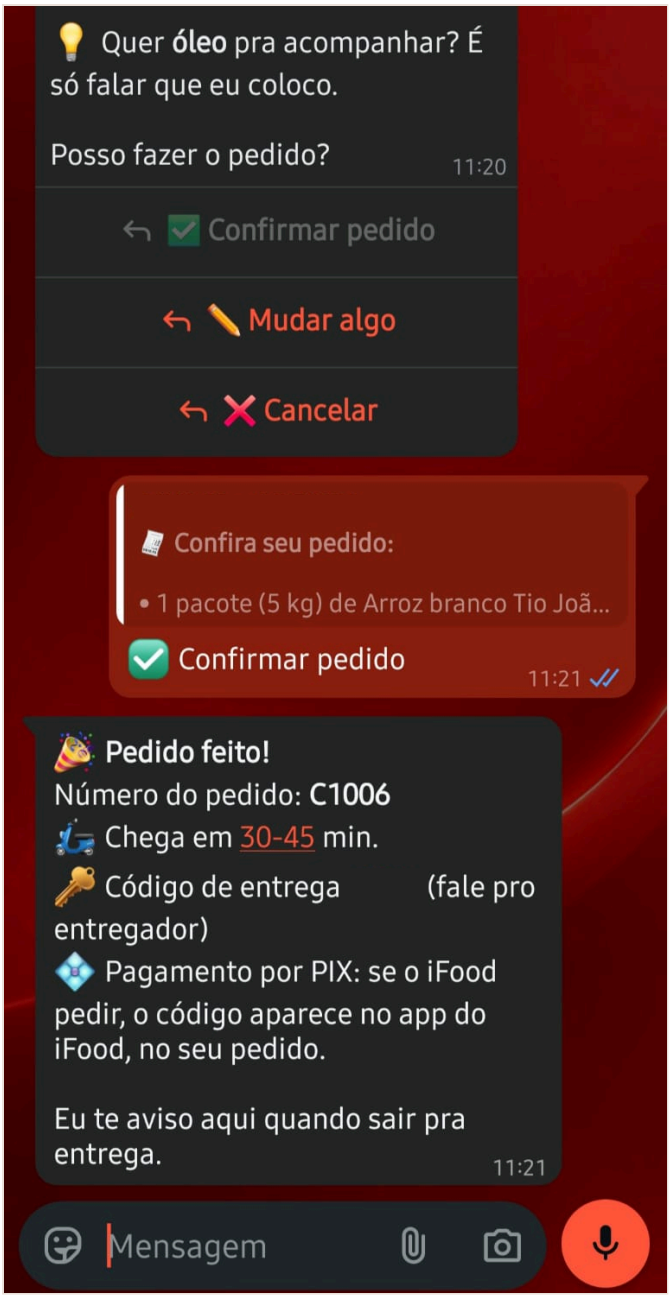
A4. Opções com foto. Carrossel de cards (até 10 por vez), cada um com foto, nome, preço e loja e o botão **Quero esse**; embaixo, ➕ **Mais opções**. "Gostou de algum? Toque em *Quero esse* 👆"



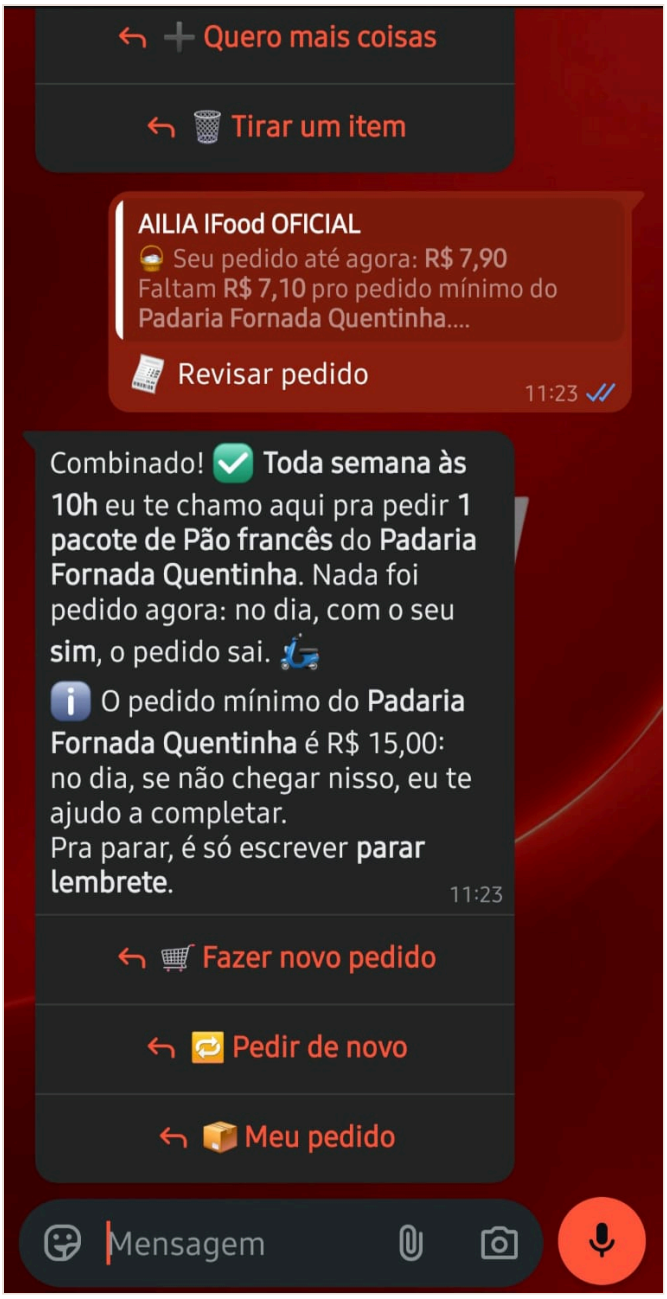
A5. Resumo e confirmação. Itens, entrega (prazo e taxa), pagamento, endereço e total, com ✅ **Confirmar pedido**, ✏️ **Mudar algo** e ❌ **Cancelar**.

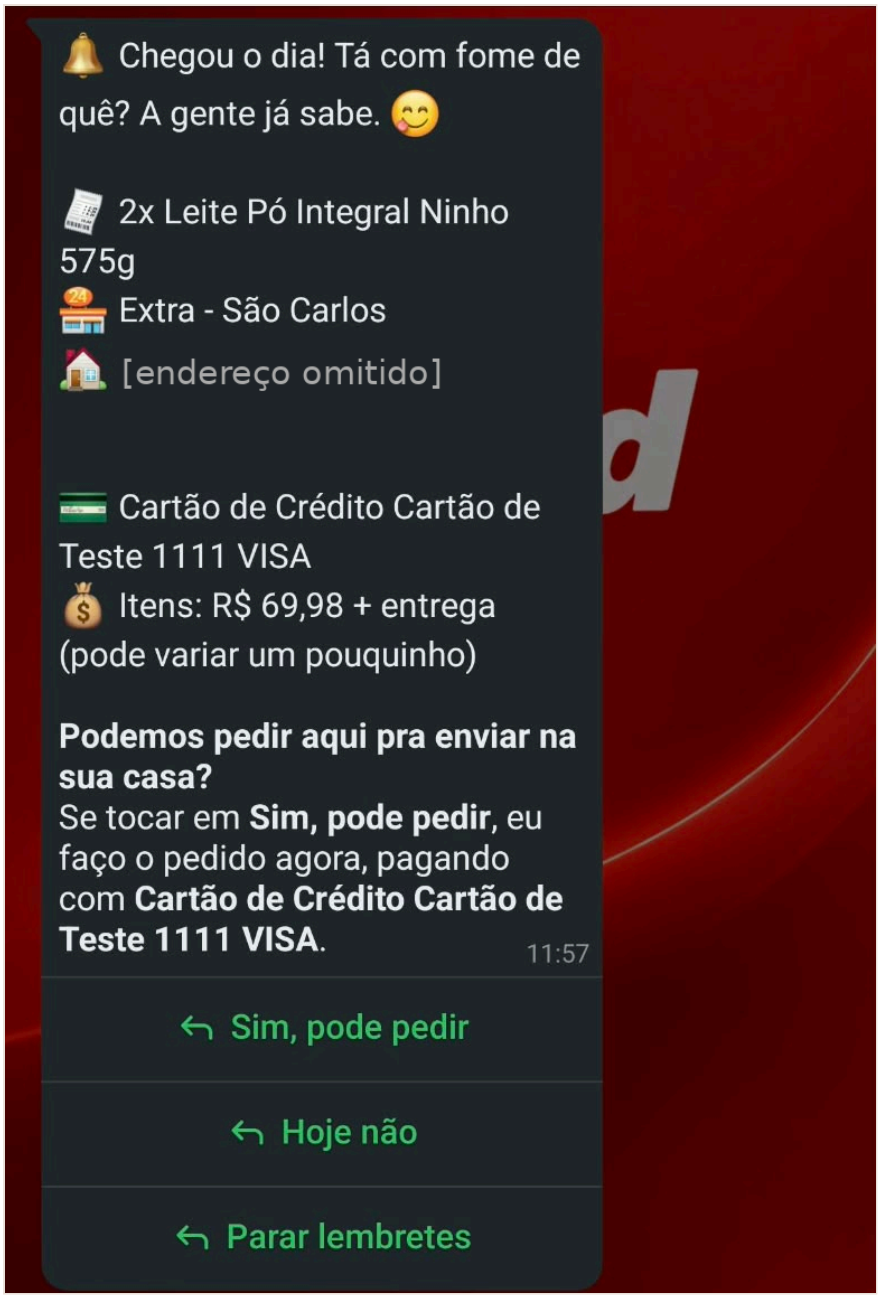


A6. Pedido feito e acompanhamento. "🎉 *Pedido feito!*" e uma mensagem por mudança de status, sempre dizendo qual pedido é (loja, itens e valor).



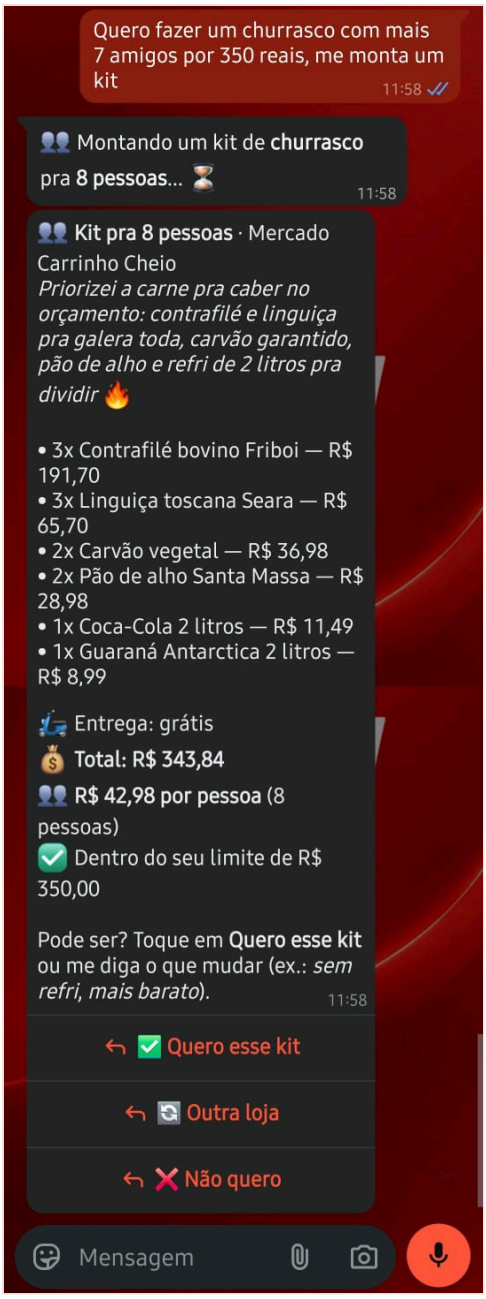
A7. Lembrete de compra recorrente. "🔔 Chegou o dia! Tá com fome de quê? A gente já sabe. 😊" com itens, loja, endereço, pagamento e "💰 Itens: R$ 69,98 + entrega (pode variar um pouquinho)" (depois do 1º pedido, "Da última vez"), e "*Podemos pedir aqui pra enviar na sua casa?*". Botões **Sim, pode pedir**, **Hoje não**, **Parar lembretes**.


↩ ➕ Quero mais coisas
↩ 🗑️ Tirar um item
AILIA IFood OFICIAL
🍔 Seu pedido até agora: R$ 7,90
Faltam R$ 7,10 pro pedido mínimo do Padaria Fornada Quentinha....
🧾 Revisar pedido
11:23
Combinado! ✅ Toda semana às 10h eu te chamo aqui pra pedir 1 pacote de Pão francês do Padaria Fornada Quentinha. Nada foi pedido agora: no dia, com o seu sim, o pedido sai. 🛵
ℹ️ O pedido mínimo do Padaria Fornada Quentinha é R$ 15,00: no dia, se não chegar nisso, eu te ajudo a completar.
Pra parar, é só escrever parar lembrete.
11:23
↩ 🛒 Fazer novo pedido
↩ 🔁 Pedir de novo
↩ 📦 Meu pedido
Mensagem

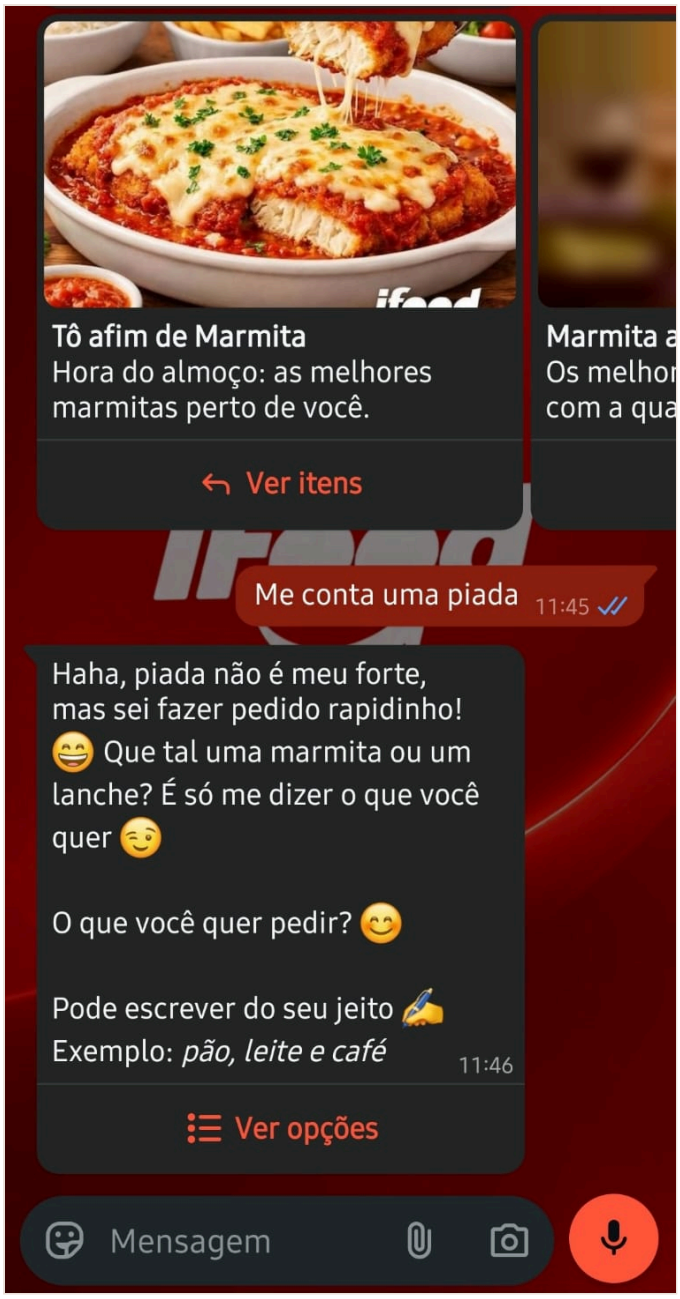


A8. Kit em grupo. "👥 Kit pra 8 pessoas · ", itens com preço, entrega, total, 👥 **R$ 42,98 por pessoa (8 pessoas)**, "✅ Dentro do seu limite de R$ 350,00" e "Pode ser? Toque em *Quero esse kit* ou me diga o que mudar (ex.: *sem refri*, *mais barato*)". Botões ✅ **Quero**

esse kit, 🔄 **Outra loja**, ❌ **Não quero**.



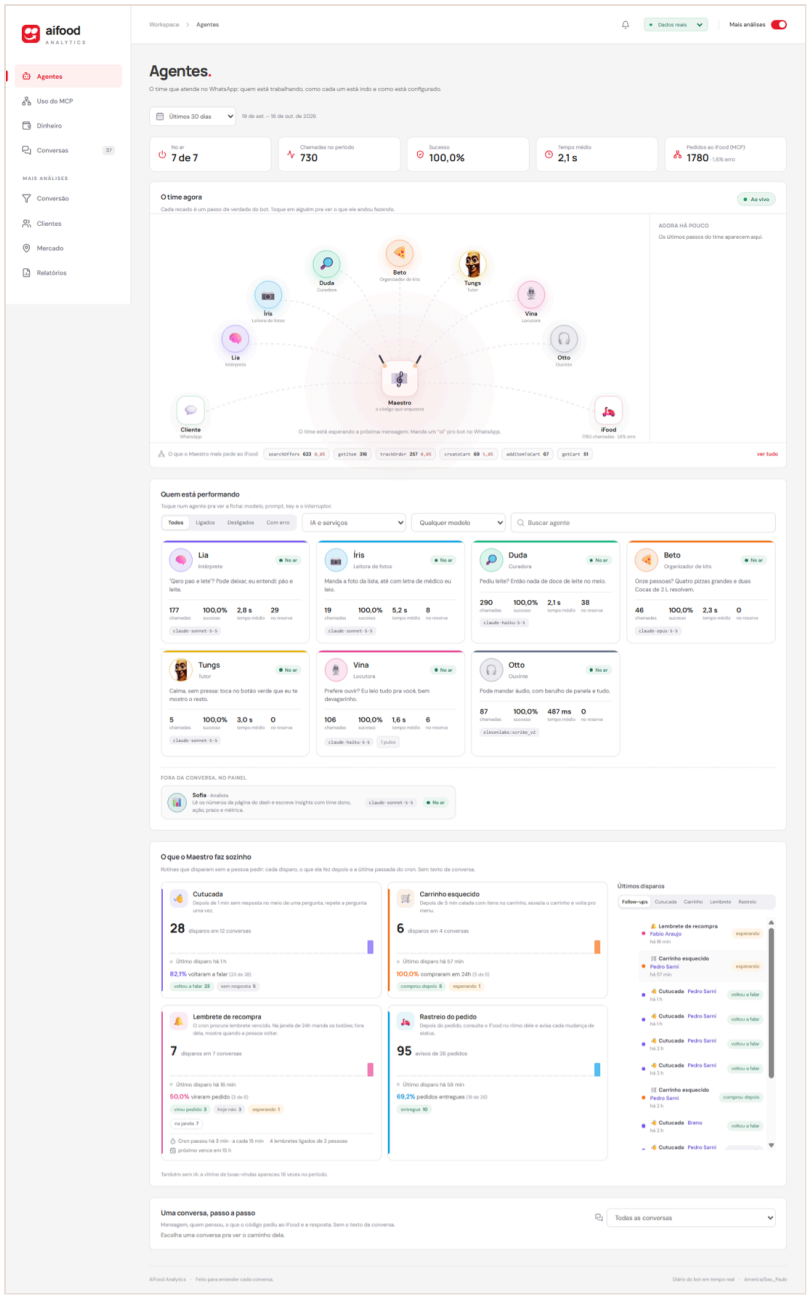
A9. Ajuda e papo fora do assunto. "Eu faço pedidos no iFood pra você. 🛒" com os 4 passos e "Nada é comprado sem você confirmar." Papo fora do assunto ("Me conta uma piada") recebe uma recusa leve da própria IA e volta ao pedido: "Haha, piada não é meu forte, mas sei fazer pedido rapidinho! 😄 Que tal uma marmita ou um lanche?...", seguido de "O que você quer pedir? Pode escrever do seu jeito ✍️ Exemplo: pão, leite e café" e o botão **Ver opções**. Sem a IA, a resposta fixa é "Eu só sei fazer pedidos no iFood 🙂". Os 4 passos acima são a resposta de "ajuda", sem print.

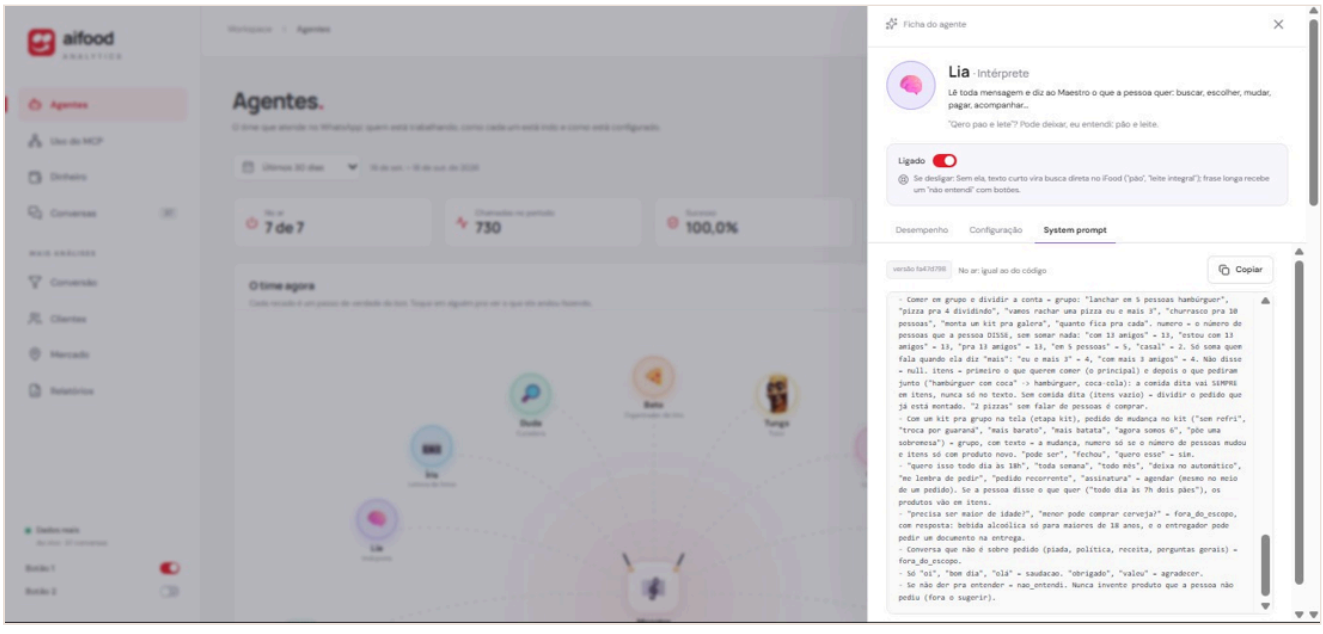


Parte B — O dash do time

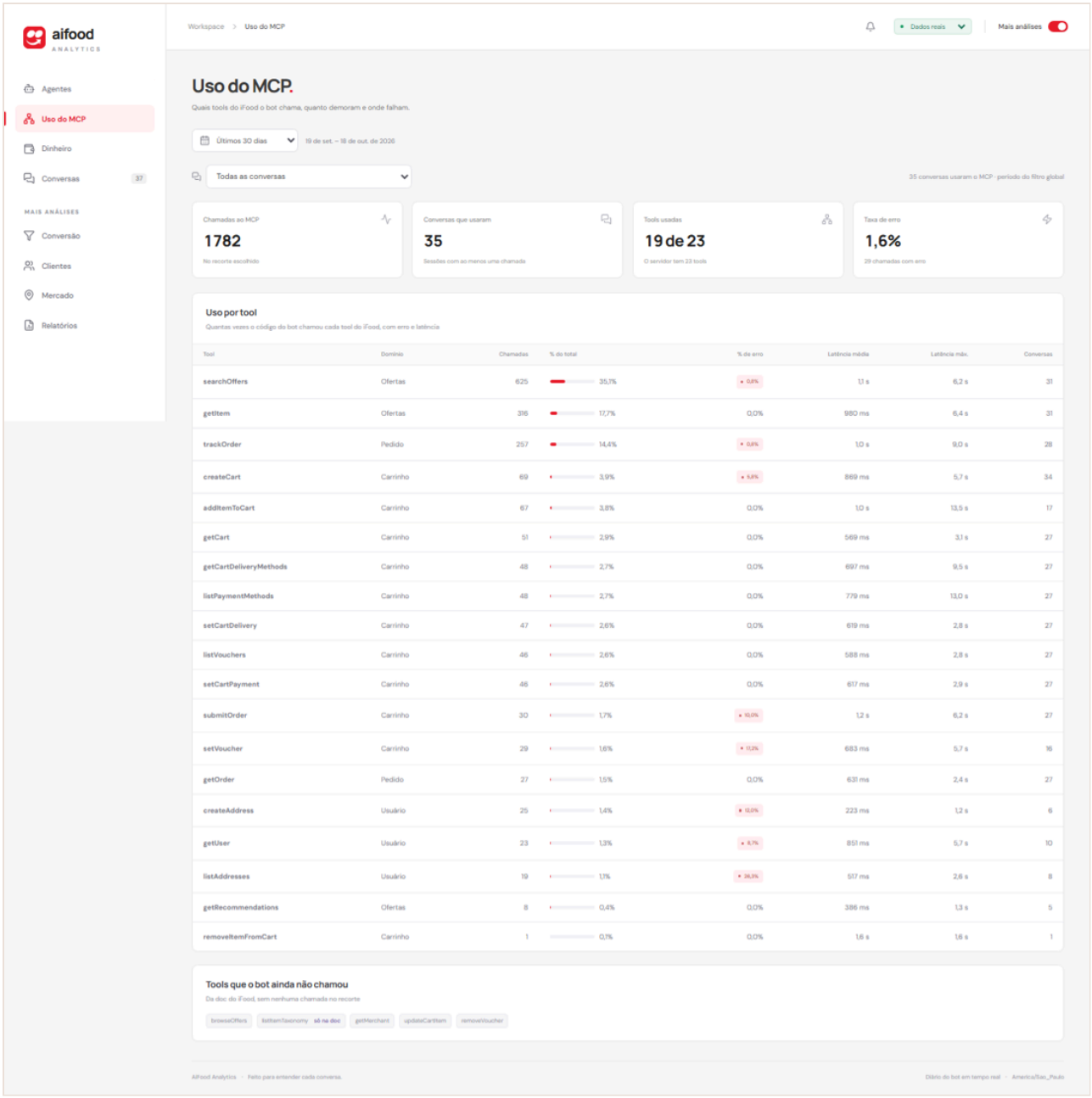
Identidade: vermelho `#EA1D2C`, branco e cinza. Seletor de fonte no topo: **Dados reais** (diário do bot ao vivo) e **Demonstração** (conversas simuladas, sempre com esse rótulo). Menu: **Agentes → Uso do MCP → Dinheiro → Conversas**, e "Mais análises" (Conversão, Clientes, Mercado, Relatórios).

B1. Agentes. Palco ao vivo (cada passo vira um recado entre cliente, Maestro, agente e iFood); time com crachá (Lia, Íris, Duda, Beto, Tungs, Vina, Otto; a Sofia fica fora do palco, no painel); ficha de cada agente com modelo, prompt, keys mascaradas e interruptor; rotinas automáticas (cutucada, carrinho esquecido, lembrete, rastreio).

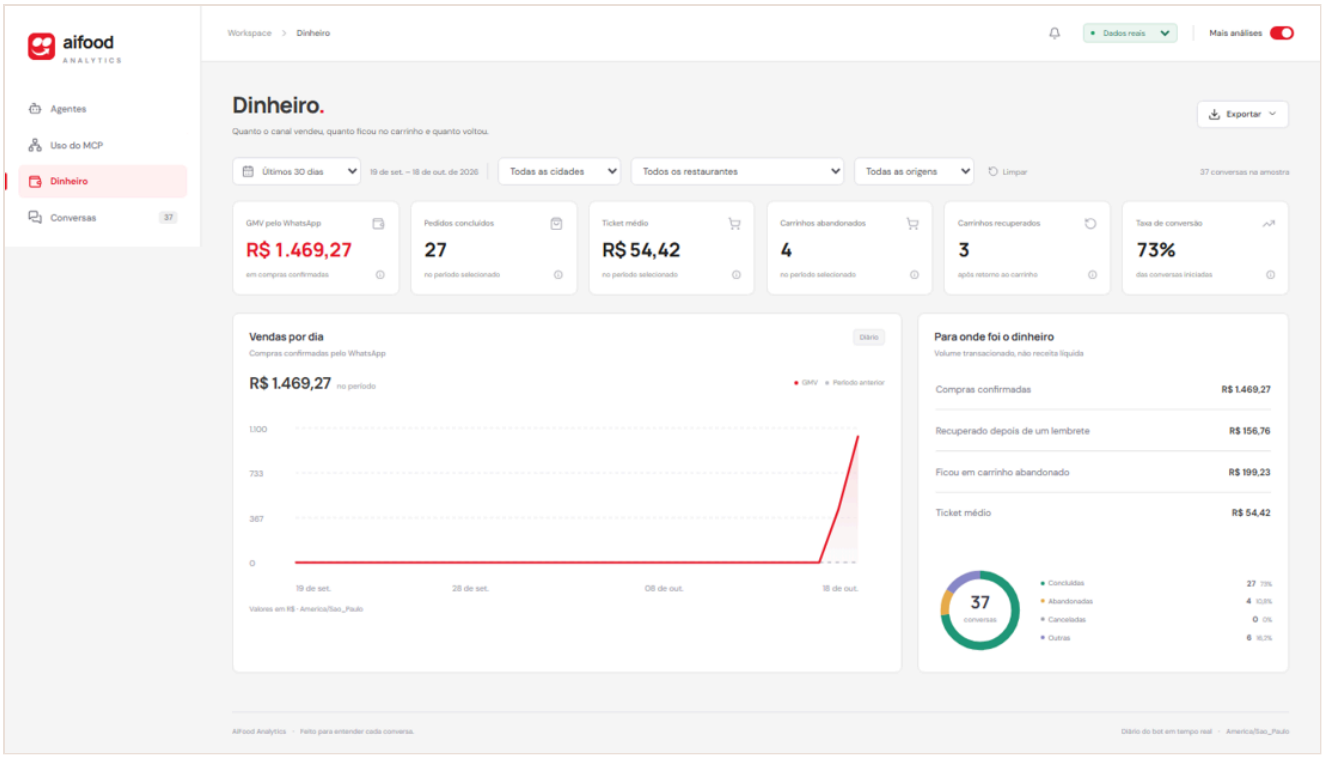

aifood
ANALYTICS
Agentes
Uso do MCP
Dinheiro
Conversas
MAIS ANÁLISES
Conversão
Clientes
Mercado
Relatórios
Workspace > Agentes
Dados reais
Mais análises
Agentes.
O time que atende no WhatsApp: quem está trabalhando, como cada um está indo e como está configurado.
Últimos 30 dias
No ar
7 de 7
Chamadas no período
730
Sucesso
100,0%
Tempo médio
2,1 s
Pedidos ao iFood (MCP)
1780
O time agora
Cada recado é um passo de verdade do bot. Toque em alguém pra ver o que ele andou fazendo.
Ao vivo
Duda
Beto
Tungs
Íris
Vina
Lia
Otto
Maestro
Cliente
iFood
AGORA HÁ POUCO
Os últimos passos do time aparecem aqui.
O time está esperando a próxima mensagem. Manda um "oi" pro bot no WhatsApp.
ver tudo
Quem está performando
Toque num agente pra ver a ficha: modelo, prompt, key e o interruptor.
Todos
Ligados
Desligados
Com erro
IA e serviços
Qualquer modelo
Buscar agente
Lia
Intérprete
No ar
"Qero pao e lete"? Pode deixar, eu entendi: pão e leite.
177
100,0%
2,8 s
29
Íris
Leitora de fotos
Manda a foto da lista, até com letra de médico eu leio.
19
100,0%
5,2 s
8
Duda
Curadora
Pediu leite? Então nada de doce de leite no meio.
290
100,0%
2,1 s
38
Beto
Organizador de kits
Onze pessoas? Quatro pizzas grandes e duas Cocas de 2 L resolvem.
46
100,0%
2,3 s
0
Tungs
Tutor
Calma, sem pressa: toca no botão verde que eu te mostro o resto.
5
100,0%
3,0 s
0
Vina
Locutora
Prefere ouvir? Eu leio tudo pra você, bem devagarinho.
106
100,0%
1,6 s
6
Otto
Ouvinte
Pode mandar áudio, com barulho de panela e tudo.
87
100,0%
487 ms
0
FORA DA CONVERSA, NO PAINEL
Sofia
Analista
Lê os números da página do dash e escreve insights com time dono, ação, prazo e métrica.
O que o Maestro faz sozinho
Rotinas que disparam sem a pessoa pedir: cada disparo, o que ela fez depois e a última passada do cron. Sem texto da conversa.
Cutucada
Depois de 1 min sem resposta no meio de uma pergunta, repete a pergunta uma vez.
28
disparos em 12 conversas
82,1% voltaram a falar
Carrinho esquecido
Depois de 5 min calada com item no carrinho, escuta o carrinho e volta pro menu.
6
disparos em 4 conversas
100,0% compraram em 24h
Lembrete de recompra
O cron procura lembrete vencido. Na janela de 24h manda os bobos; fora dela, mostra quando a pessoa volta.
7
disparos em 7 conversas
50,0% viram pedido
Rastreio do pedido
Depois do pedido, consulta o iFood no ritmo dele e avisa cada mudança de status.
95
avisos de 26 pedidos
69,2% pedidos entregues
Últimos disparos
Follow-ups
Cutucada
Carrinho
Lembrete
Rastreio
Uma conversa, passo a passo
Mensagem, quem pensou, o que o código pediu ao iFood e a resposta. Sem o texto da conversa.
Escolha uma conversa pra ver o caminho dela.
Todas as conversas

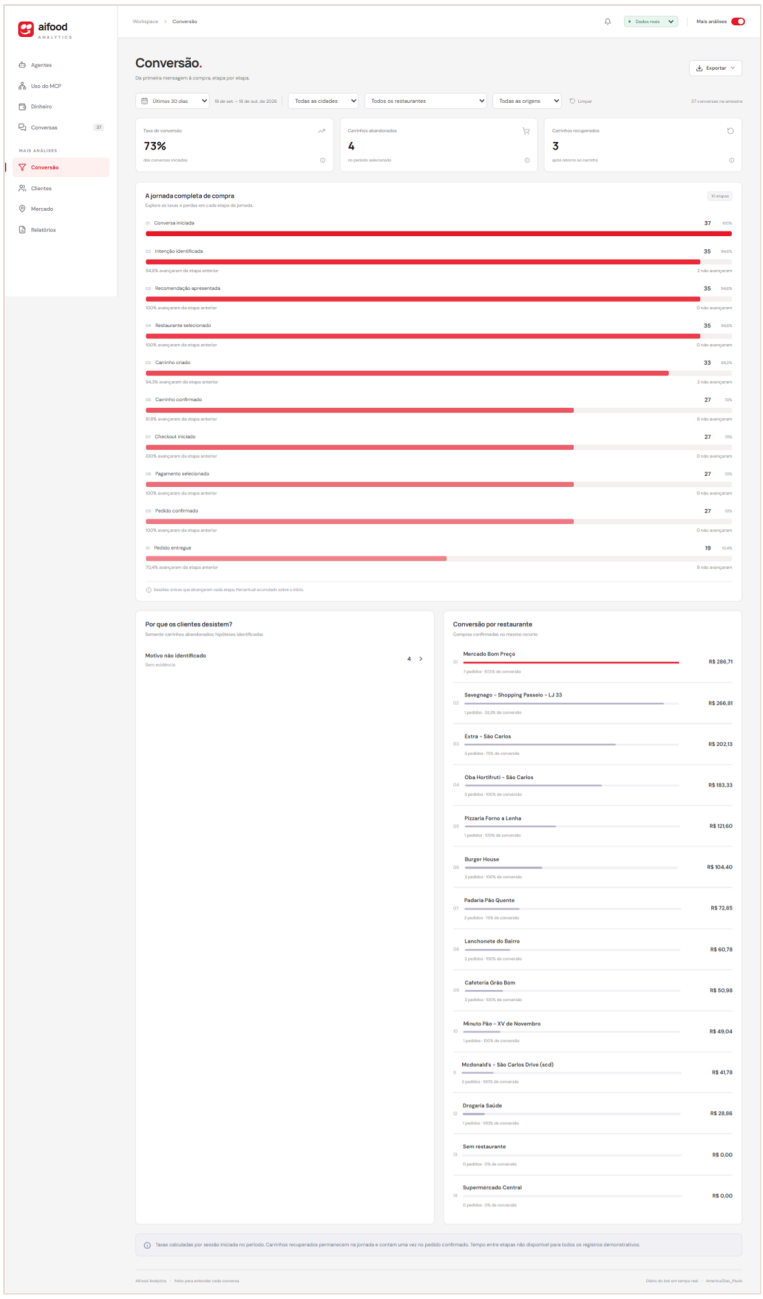

Ficha do agente
Lia · Intérprete
Lê toda mensagem e diz ao Maestro o que a pessoa quer: buscar, escolher, mudar, pagar, acompanhar...
"Qero pao e lete"? Pode deixar, eu entendi: pão e leite.
Ligado
Se desligar: Sem ela, texto curto vira busca direta no iFood ("pão", "leite integral"); frase longa recebe um "não entendi" com botões.
Desempenho
Configuração
System prompt
No ar: igual ao do código
Copiar
- Comer em grupo e dividir a conta = grupo: "lanchar em 5 pessoas hambúrguer", "pizza pra 4 dividindo", "vamos rachar uma pizza eu e mais 3", "churrasco pra 10 pessoas", "monta um kit pra galera", "quanto fica pra cada". numero = o número de pessoas que a pessoa DISSE, som somar nada: "com 13 amigos" = 13, "estou com 13 amigos" = 13, "pra 13 amigos" = 13, "em 5 pessoas" = 5, "casal" = 2. Só soma quem fala quando ela diz "mais": "eu e mais 3" = 4, "com mais 3 amigos" = 4. Não disse = null. itens = primeiro o que querem comer (o principal) e depois o que pediram junto ("hambúrguer com coca" -> hambúrguer, coca-cola): a comida dita vai SEMPRE em itens, nunca só no texto. Sem comida dita (itens vazio) = dividir o pedido que já está montado. "2 pizzas" sem falar de pessoas é comprar.
- Com um kit pra grupo na tela (etapa kit), pedido de mudança no kit ("sem refri", "troca por guaraná", "mais barato", "mais batata", "agora somos 6", "põe uma sobremesa") = grupo, com texto = a mudança, numero só se o número de pessoas mudou e itens só com produto novo. "pode ser", "fechou", "quero esse" = sim.
- "quero isso todo dia às 18h", "toda semana", "todo mês", "deixa no automático", "me lembra de pedir", "pedido recorrente", "assinatura" = agendar (mesmo no meio de um pedido). Se a pessoa disse o que quer ("todo dia às 7h dois pães"), os produtos vão em itens.
- "precisa ser maior de idade?", "menor pode comprar cerveja?" = fora_do_escopo, com resposta: bebida alcoólica só para maiores de 18 anos, e o entregador pode pedir um documento na entrega.
- Conversa que não é sobre pedido (piada, política, receita, perguntas gerais) = fora_do_escopo.
- Só "oi", "bom dia", "olá" = saudacao. "obrigado", "valeu" = agradecer.
- Se não der pra entender = nao_entendi. Nunca invente produto que a pessoa não pediu (fora o sugerir).

B2. Uso do MCP. Cada chamada ao iFood: tool, tempo, sucesso ou erro, e as tools da doc nunca chamadas.

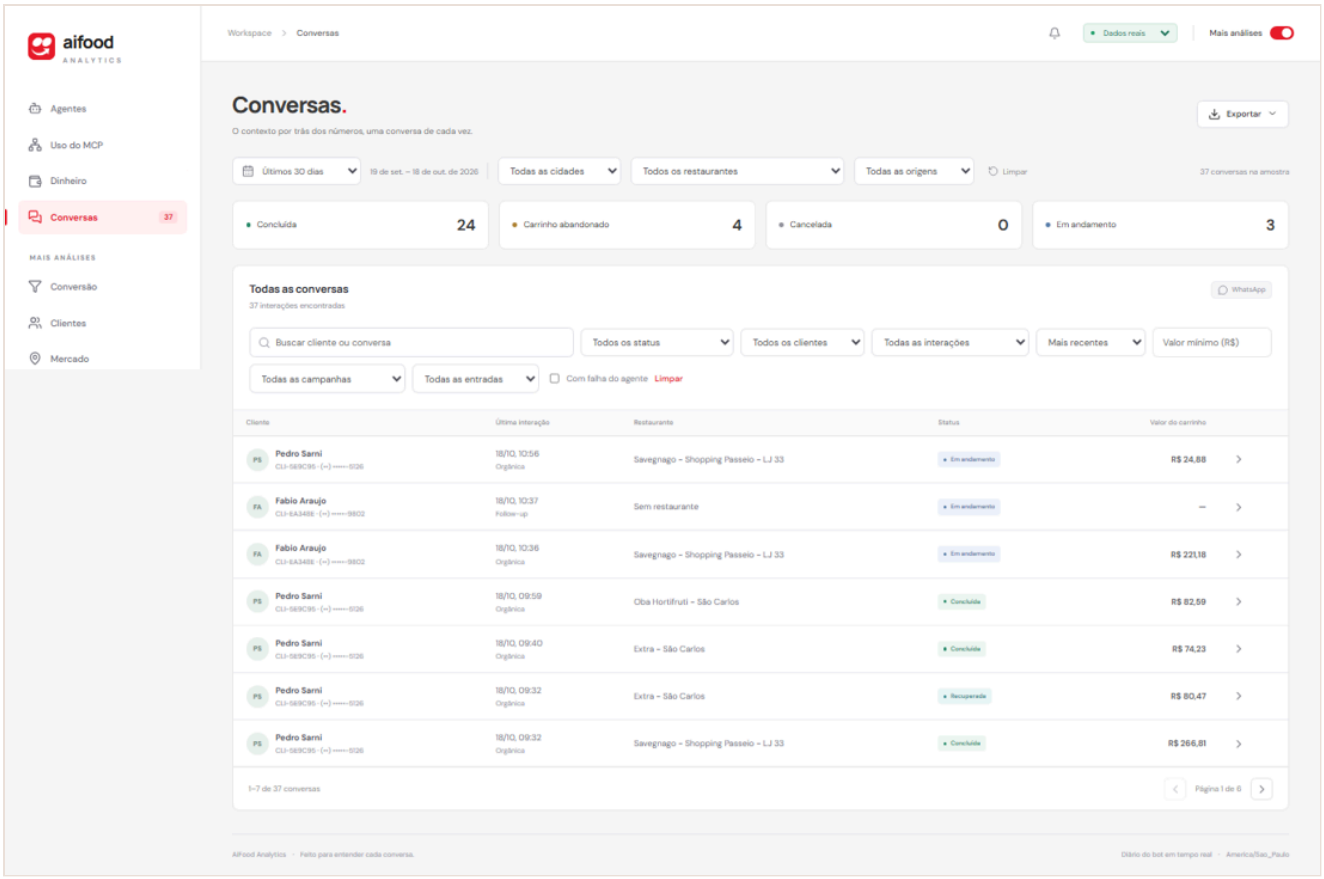


B3. Dinheiro e Conversão. Vendas, funil (recomendação → seleção → carrinho → checkout → pedido → entrega), abandono e recuperação.


aifood
ANALYTICS
Agentes
Uso do MCP
Dinheiro
Conversas
37
Workspace > Dinheiro
Dados reais
Mais análises
Dinheiro.
Quanto o canal vendeu, quanto ficou no carrinho e quanto voltou.
Exportar
Últimos 30 dias
19 de set. – 18 de out. de 2026
Todas as cidades
Todos os restaurantes
Todas as origens
Limpar
37 conversas na amostra
GMV pelo WhatsApp
R$ 1.469,27
em compras confirmadas
Pedidos concluídos
27
no período selecionado
Ticket médio
R$ 54,42
no período selecionado
Carrinhos abandonados
4
no período selecionado
Carrinhos recuperados
3
após retorno ao carrinho
Taxa de conversão
73%
das conversas iniciadas
Vendas por dia
Compras confirmadas pelo WhatsApp
Diário
R$ 1.469,27 no período
GMV
Período anterior
1.100
733
367
0
19 de set.
28 de set.
08 de out.
18 de out.
Valores em R$ · America/Sao_Paulo
Para onde foi o dinheiro
Volume transacionado, não receita líquida
Compras confirmadas
R$ 1.469,27
Recuperado depois de um lembrete
R$ 156,76
Ficou em carrinho abandonado
R$ 199,23
Ticket médio
R$ 54,42
37
conversas
Concluídas
27 73%
Abandonadas
4 10,8%
Canceladas
0 0%
Outras
6 16,2%
AiFood Analytics · Feito para entender cada conversa.
Diário do bot em tempo real · America/Sao_Paulo

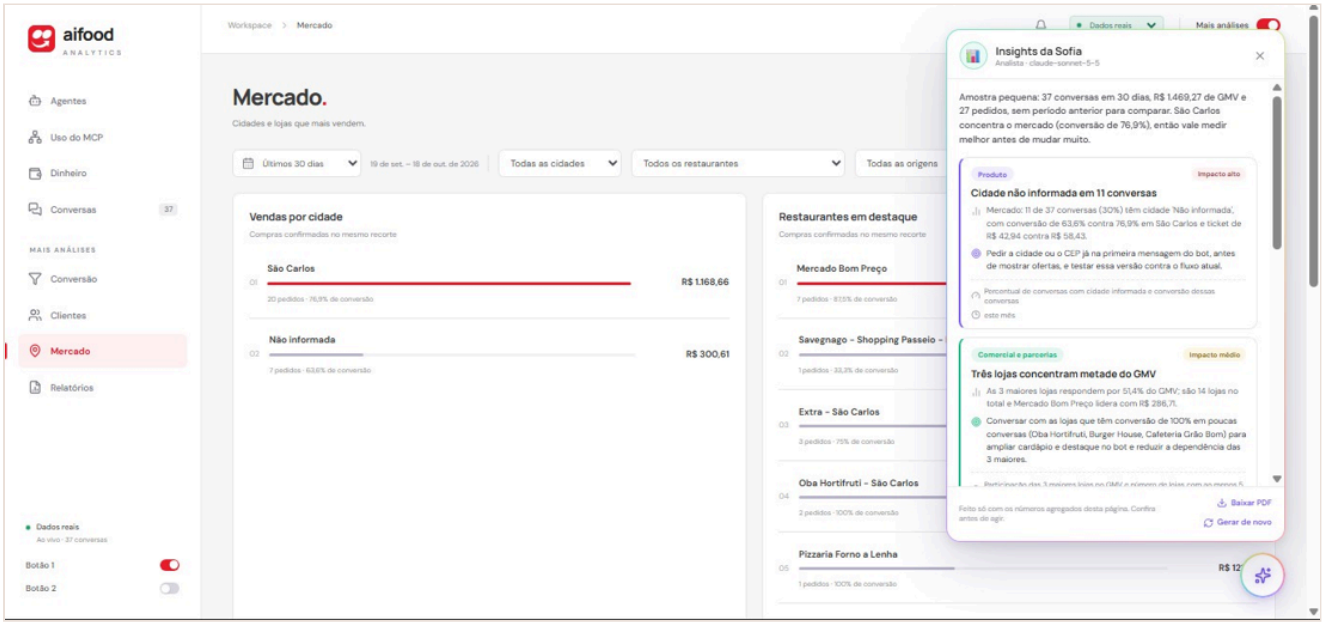


B4. Conversas. Tabela com busca e filtros; o drawer mostra a conversa com CPF e telefone mascarados.



A captura mostra a lista de conversas; ao tocar em uma conversa, abre o histórico completo ao lado (mensagens, eventos e análise).

B5. Insights com IA. Botão flutuante em Conversão, Clientes e Mercado; a Sofia devolve 3 a 5 insights (time dono, evidência, ação, impacto, prazo, métrica) e exporta em PDF.



Como ver ao vivo. WhatsApp: demonstração ao vivo durante o pitch; contato com o time sob pedido. Dash: roda local (`npm run dev` em `Frontend/Rascunho_Dash`, com `python -m src.painel.servidor` na raiz).

05 Documento Técnico Resumido

Detalhe completo em `docs/como-funciona.md`.

5.1 Tese: "a IA entende, o código garante"

Os modelos só transformam a mensagem (texto, áudio, foto) em JSON de formato fixo. Quem decide o próximo passo, chama o iFood, faz contas e conduz o pedido é código determinístico. A confirmação de compra nunca parte de um modelo.

5.2 Arquitetura

```
WhatsApp ──► Meta Cloud API ──► webhook /whatsapp (Modal, responde 200 na hora)
                                        │ fila por pessoa
                                        ▼
                                   rodada (junta a rajada)
                                        ▼
                              Bot.processar  (src/bot/fluxo.py, "Maestro")
                                ├─ botão → caminho sem IA
                                ├─ texto/áudio/foto → regras rápidas → agente de IA → intenção JSON
                                └─ src/bot/ifood.py → cliente MCP (src/shared/ifood_mcp.py) ou mock com a mesma interface
                                        ▼
                                respostas com botões/cards (Graph API)
   funções de fundo: rastreio, lembretes (cron 15 min), cutucada/abandono, ponte_mcp
   diário de eventos (modal.Dict) ──► servidor local (127.0.0.1) ──► dash React
```

Camada	Tecnologia
Backend	Python no Modal (webhook, funções de fundo, cron)
Estado	`modal.Dict` por pessoa (conversa, tokens, lembretes, configuração)
Canal	WhatsApp Cloud API oficial da Meta
IA de texto e foto	Claude (Sonnet, Haiku ou Opus 5.5, conforme o agente) na frente, Gemini de reserva, OpenRouter por último
Áudio	ElevenLabs Scribe (ouvir) e ElevenLabs (falar); Gemini ouve se o Scribe falhar
Dash	React + React Router + Vite, servidor local só em 127.0.0.1

5.3 Agentes

8 agentes, um arquivo cada em `src/agentes/`, com ficha, plano B e fila de modelos própria. Os agentes não se chamam entre si; o código orquestra. Cada um pode ser desligado ou trocar de modelo pelo dash, sem deploy.

Agente	Papel	1º da fila hoje	Plano B sem IA
Lia	entende a mensagem (intenção, itens, quantidade)	Claude Sonnet 5.5	regras rápidas; texto curto vira busca direta, frase longa recebe "não entendi" com botões
Íris	lê fotos (comida, lista, produto, outra coisa)	Claude Sonnet 5.5	pede para escrever
Duda	filtra e ordena as ofertas pelo que foi pedido	Claude Haiku 5.5	filtro por palavra no nome
Beto	composição do kit em grupo	Claude Opus 5.5 (prazo 20 s)	`kit_padrao` pela receita
Tungs	tutor (como usar o bot)	Claude Sonnet 5.5	textos fixos
Vina	roteiro da resposta por áudio	Claude Haiku 5.5	resposta em texto
Otto	transcrição de áudio	ElevenLabs Scribe	Gemini ouve
Sofia	insights no dash (fora da conversa)	Claude Sonnet 5.5	—

5.4 Uso do MCP

Usamos **19 das 23 tools** que o servidor expõe (a doc fala em 24; `listItemTaxonomy` não existe no servidor), contadas por chamada real em `src/bot/ifood.py`. Não usamos `browseOffers`, `getMerchant`, `updateCartItem` e `removeVoucher`.

Etapa	Tools
Conta e endereço	`getUser`, `listAddresses`, `createAddress` (dois passos)
Descoberta	`searchOffers` (só o termo; resto em refinements), `getRecommendations` (`RECENT_ORDERS`), `getItem`
Carrinho	`createCart` (itens ou `orderId`), `addItemToCart`, `removeItemFromCart`, `getCart`
Checkout	`getCartDeliveryMethods`, `setCartDelivery`, `listPaymentMethods`, `setCartPayment`, `listVouchers`, `setVoucher`
Pedido	`submitOrder(userConsent=true)`, `getOrder`, `trackOrder` (no ritmo de `agentHints` até `pollingStop`)

Decisões em cima do MCP (configuração e dificuldades na parte 06):

- **Uma busca, várias páginas:** `searchOffers` com `pageSize` 25; "Mais opções" pagina localmente antes de pedir a próxima página.
- **Melhor loja:** busca cada item e escolhe a loja com todos os itens e menor total.
- **Opções do item:** o mapeamento para o carrinho é ambíguo; o bot tenta um formato e cai no outro se o iFood recusar.
- **Recorrência nossa:** guardamos um modelo do pedido e refazemos com `createCart`.
- **Proteção do gateway:** freio de 0,35 s entre chamadas por container, disjuntor de 300 s ao ver o 403 do Akamai, container barrado se aposenta e a ponte refaz a chamada de outro IP.

- **Contingência de apresentação:** com o "modo palco" armado (desligado por padrão), a chamada real tem prazo de 10 s e uma tentativa; na primeira queda o mock assume e a mesma chamada é refeita nele.

5.5 Decisões técnicas

- **Regras antes da IA:** "sim", "não", números, "menu", "cancelar", "já pode passar no pix" são resolvidos sem modelo.
- **JSON garantido:** esquema fixo de intenção (`ESQUEMA` em `src/bot/interpretar.py`); resposta fora do esquema cai no plano B.
- **Confirmação no código:** `submitOrder` só depois de um dos gatilhos da seção 1.3; trava contra toque duplo.
- **Dinheiro só no código;** "por pessoa" arredondado para cima no centavo.
- **Pagamento só PIX e vales** (`PIX`, `MEAL_VOUCHER`, `FOOD_VOUCHER`).
- **Botões com identificador:** botão antigo gera um aviso só; o "sim" de uma foto antiga não vale para a nova.
- **Configuração sem deploy:** interruptores no armazém via `scripts/modo.py` (mock, palco, lojas fechadas, lembrete rápido, cutucada).

5.6 Segurança e privacidade

CPF validado só no código, nunca enviado a modelo, enviado ao iFood só no fechamento (ou quando a loja exige ao adicionar o item, ver D12). Foto não é aberta na etapa do CPF. O diário grava CPF mascarado, sem token nem link de login; dash só em 127.0.0.1. Keys só nos secrets do Modal; o dash mostra os 4 últimos caracteres. "Sair" revoga o login iFood; "apagar meus dados" apaga o que guardamos.

5.7 Qualidade

- **545 testes** no back (pytest, contra o mock com a mesma interface do MCP) e **54** no dash (Vitest), todos passando em 18/10 após 10h26.
- QA manual no WhatsApp (mock e MCP real), registrado em `docs/qa-18-10.md`.
- Colaboração por claims (`.coord/claims/`), hooks de autoria e de arquivos protegidos.

---

06 Feedback sobre os MCPs

Servidor: gateway `consumer-mcp.ifood.com.br/marketplace` (4 domínios). Doc de referência: portal v0.27.0, consultado em 17/10 (resumo em `regras da api/ifood-mcp-developer-portal.json`).

6.1 Como usamos e testamos

Usamos o MCP de dois jeitos, e as marcas valem para os dois:

1. **QA manual de 17/10** (conta de teste, uma chamada por vez): cobriu User, Discovery, Cart e Order Tracking com sucesso, entradas inválidas, limites e erros. Regras: sem `submitOrder`, sem cartão, nenhum token ou secret registrado. O entregável 06 original fala em ~70 achados; o log completo não está no repo, só o resumo em `docs/contexto-evento.md` §8 e o próprio 06. Os horários desse log são ordem das chamadas, não hora exata.
2. **O bot em produção**, 17/10 à tarde até 18/10 de manhã, alternando entre o MCP real e o mock (ver Anexo A). Aqui apareceram o bloqueio do Akamai, a latência, o CPF no item, os totais com taxa e o rastreio de um pedido real.

6.2 Como configuramos (passo a passo)

Quando	Passo	O que aconteceu
17/10 14h40–15h03	Cadastro no portal	Portal fora do ar; mentores avisaram que estavam resolvendo [testado]
~15h10	1º agente, "HackFood Agent" (Davi)	Redirect só em localhost (porta 8080, rota `/callback`); escopos de acesso: Dados de usuário, Descoberta e Criação de pedidos. Aprovado em menos de 1h, antes das 15h58 [testado]
15h13	Leitura da doc	Resumo da doc do portal e do llms.txt no repo, com as divergências já anotadas
~16h05	2º agente, "Compras pelo WhatsApp" (Fabio)	Redirects: a rota `/ifood/callback` do backend publicado no Modal e o mesmo localhost. Mesmos 3 escopos. Necessário porque o 1º só tinha localhost, que não serve para login pelo celular
16h18	Primeira versão do cliente	OAuth com escopo padrão `openid`; cliente JSON-RPC sobre HTTP no gateway
16h30	`POST /token` do 1º agente	`invalid_client` com o secret recém-gerado (post, Basic e sem secret). Causa: o secret estava errado; não era problema do iFood [testado]
até 17h05	`POST /token` do 2º agente	Com um `code` falso, devolve `invalid_grant`: a autenticação do cliente passa por `client_secret_post` e por `client_secret_basic` [testado]
até 17h09	Escopos	`openid` não é permitido para o nosso cliente; a string de escopos ficou só `mcp.*` [testado]
17h32–17h37	Jornada de login	Login web direto no `id.ifood.com.br`, sem o OneLink do app, que exige redirect por Universal/App Link
~21h05	Mensagem da organização	Com callback em localhost o iFood confere só o path, não a porta; a edição da callback URI está bloqueada no portal. Por isso o nome do app, o label, o workspace e a região do Modal ficaram congelados
17/10 em diante	Contas de teste	Conta de teste do portal (e-mail `@test.ifood.com.br`, OTP exibido no portal) no QA e no bot. Quantas contas criamos: não registrado

Configuração final (código em `src/shared/ifood_mcp.py`):

- **OAuth 2.1:** Authorization Code + PKCE S256, cliente confidencial (secret só no backend, nos secrets do Modal), `state` com o `code_verifier` guardado no servidor, renovação 5 min antes de vencer, refresh rotativo (guardamos sempre o novo), `revoke` no "sair".
- **Escopos pedidos:** os amplos `mcp.user mcp.discovery mcp.cart` e os granulares de User, Discovery e Cart, inclusive `mcp.orders.read`, com override por variável de ambiente se der `invalid_scope`.
- **Host:** só o gateway `consumer-mcp.ifood.com.br/marketplace` e o emissor `id.ifood.com.br`.
- **Protocolo MCP:** `initialize` com `protocolVersion` `2025-06-18` e reuso do `Mcp-Session-Id` por 10 min, tudo deduzido da especificação do MCP, porque a doc do iFood não mostra essa parte [só na doc].
- **Ambiente:** app `hackfood-bot` no Modal, URL fixa porque está cadastrada como callback.
- **Ritmo:** no máximo ~3 chamadas/s por container; o resto é descrito em D6.

6.3 Dificuldades, uma a uma

Formato: **Doc** (o que esperávamos) · **Real** (o que aconteceu) · **Impacto** · **O que fizemos**.

D1. Portal de cadastro fora do ar [testado]

- **Real:** 17/10, 14h40 a 15h03, logo no começo do evento.
- **Impacto:** o cadastro só andou depois das 15h03; o agente foi criado ~15h10 e aprovado em menos de 1h.

D2. `invalid_client` no `/token` [testado]

- **Doc:** o Client Secret é gerado após a aprovação e aparece uma única vez [só na doc].
- **Real:** 16h30, `invalid_client` com o secret recém-gerado do 1º agente. O 2º agente passou na autenticação às 17h05. Conclusão: o secret usado estava errado. Não era bug do iFood.
- **Impacto:** das 16h30 às 17h05 sem saber se o problema era nosso ou da plataforma.
- **Dica que deixamos ao time:** conferir se o secret é do mesmo agente do `client_id` e gerar de novo se houver dúvida, porque ele some ao recarregar a página.

D3. Escopo `openid` recusado e escopos que o portal não mostra

- **Doc:** o llms.txt lista `openid` como obrigatório, além de `profile`, `email`, `phone`, `address` [só na doc]; a doc pede `mcp.orders.read` para rastrear [só na doc].
- **Real:** `openid` não é permitido para o nosso cliente; só `mcp.*` funciona [testado]. O card "Criação de pedidos" do portal mostra só `mcp.cart`, e `mcp.orders.read` não aparece em card nenhum [testado no portal]. Mesmo assim, `trackOrder` funciona: devolve `ORDER_NOT_FOUND`, não erro de acesso [testado].
- **Impacto:** a primeira versão (16h18) pedia `openid`; trocamos antes das 17h09. Como não há tool de introspecção, os escopos efetivos do token foram inferidos pelo

comportamento.
- **Sugestão:** expor os escopos do token (`grantedScopes`).

D4. Hosts e nomes divergentes entre doc e llms.txt

- **Doc:** os guias usam `consumer-mcp.ifood.com.br`; o llms.txt cita `api-mcp.ifood.com` e `dev-api-mcp.ifood.com` [só na doc]. Para auth, `id.ifood.com.br` nos guias e `auth.ifood.com.br` / `dev-auth.ifood.com.br` no llms.txt [só na doc]. O llms.txt usa escopos `user:read` / `cart:write` e `code_challenge_method=pkce`; as páginas usam `mcp.*` e `S256` [só na doc]. O quickstart da home cita `@ifood/mcp-sdk` e `mcp.ifood.dev/catalog`, que não batem com o resto [só na doc].
- **Real:** os hosts de MCP do llms.txt nem resolvem no DNS; o `consumer-mcp` responde 401 com `resource_metadata` [testado]. Só o `id.ifood.com.br` tem OIDC de verdade (authorize, token, revoke, PKCE S256); os outros devolvem 302 ou vazio [testado].
- **Impacto:** testamos host por host até achar os que respondem; o código usa só o gateway e o `id.ifood.com.br`.

D5. Registro dinâmico anunciado e inexistente; handshake MCP sem doc

- **Real:** o 401 do gateway manda o cliente "re-register automatically", mas não existe `registration_endpoint`; credencial só pelo portal [testado]. A doc não mostra `initialize`, `Mcp-Session-Id` nem `protocolVersion` [só na doc].
- **Impacto:** implementamos o handshake pela especificação do MCP, sem exemplo do iFood.

D6. Bloqueio 403 do Akamai por rajada (o que mais custou)

- **Doc:** os Termos citam rate limit e quotas, mas os valores não são publicados [só na doc].
- **Real, 1ª vez (17/10 ~18h30):** ~50 chamadas em poucos minutos, vindas dos nossos scripts de teste, e o firewall barrou o IP de saída do Modal por ~20 min: `403 Access Denied` em página HTML do Akamai, até sem token, fora do formato MCP e sem `Retry-After` [testado]. O bot inteiro ficou fora.
- **Real, 2ª vez (18/10 ~02h40):** de novo o 403 no container do bot. A `scripts/sondar_403.py`, saindo de outro IP do Modal com os mesmos tokens, recebeu 200 [testado]. Pelo que observamos, o bloqueio é por IP de saída, não por token.
- **Real, medição (18/10 ~07h29):** 8 containers do Modal em 5 IPs, 1 barrado; os 4 do bot no ar estavam livres [testado]. Um disjuntor compartilhado, feito supondo um IP só, fazia um container azarado parar todos os outros por 5 min.
- **Real, MCP mudo (18/10, antes das 06h20):** chamadas que não devolviam nem erro, registrado no código do modo palco [testado]. Quantas vezes e por quanto tempo: não registrado.
- **Impacto:** para quem hospeda um agente em nuvem (IP compartilhado e rotativo), o bloqueio sem aviso derruba todos os usuários de uma vez. Foi o motivo de o bot rodar boa parte da noite no mock (ligado às ~21h25 de 17/10 "porque o iFood real cai muito"). A decisão de demo em mock ou no real ficou em aberto (item H3 do `docs/estado.md`).

- **O que fizemos, em ordem:**
  1. 18h41: sonda para saber quem o Akamai está barrando (`scripts/sondar_403.py`).
  2. 18h53: freio de 0,35 s entre chamadas (~3/s) e disjuntor de 300 s ao ver o 403; 403 nunca entra em retry, porque insistir estica o bloqueio.
  3. 19h17: disjuntor coletivo (compartilhado pelo armazém) e botão de pânico do modo demo.
  4. 18/10 02h51: `scripts/destravar_ifood.sh`, redeploy limpo (containers novos, mesma URL), disjuntor zerado e webhook conferido, em ~20 s.
  5. 06h20: modo palco (desligado por padrão): prazo de 10 s, uma tentativa e, na primeira queda (timeout, rede, HTTP, firewall), o catálogo de reserva assume e a mesma chamada é refeita nele. Fora do palco, queda é erro de verdade.
  6. 07h29: disjuntor só local; o container que vê o 403 se aposenta (o Modal abre outro, em outro IP) e a chamada vai por uma ponte (função separada, outra máquina) antes de desistir.
  7. 08h20: a troca para o catálogo de reserva leva carrinho, produto da vez e card de antes pelo nome; a pessoa não vê.
- **Sugestão:** responder `429` com `Retry-After` dentro do envelope MCP; documentar o limite por cliente ou por token de usuário, não por IP.

D7. Latência alta para um chat [testado]

- **Real (medido no diário em 18/10 ~08h30):** kit em grupo com o iFood real levou 47 s e 60 s, com ~15 chamadas de `searchOffers`/`getItem` de ~2,5 s cada; o fechamento faz 7 chamadas em fila (~30 s), com `getCartDeliveryMethods` sozinho em 9,5 s.
- **Impacto:** com o iFood lento, a pessoa tocou "Finalizar" 3 vezes e recebeu 3 resumos (corrigido às 04h52 com um "⏳ Fechando com a loja" antes das chamadas).
- **O que fizemos:** `getItem` e buscas do kit em paralelo, com o freio segurando o ritmo; um filtro de relevância só para todas as buscas.

D8. `cartItemId` muda a cada escrita no carrinho [testado]

- **Doc:** "salvar o `cartItemId` retornado" e usá-lo em `updateCartItem`/`removeItemFromCart` [só na doc].
- **Real:** depois de `addItemToCart`, `updateCartItem`, `removeItemFromCart`, `setCartDelivery`, `setCartPayment` ou `setVoucher` (o 06 original inclui também `removeVoucher`), as linhas sem promoção ganham novo `cartItemId`.
- **Impacto:** quem guarda o ID e depois remove recebe "O item não está mais disponível" e diz ao cliente que o produto acabou.
- **Reprodução:** carrinho com 2 linhas, adicionar um item, chamar `removeItemFromCart` com o ID devolvido antes.
- **Sugestão:** ID estável por linha, ou `previousCartItemId` e um erro `CART_ITEM_NOT_FOUND` com dica "chame `getCart`".

D9. Erros sem padrão [testado]

- **Doc:** fala em 3 formatos (User/Discovery, Cart, Order Tracking) e admite a divergência [só na doc].
- **Real:** 4 formatos JSON (User, Discovery, Cart, Tracking) e 3 formas de texto cru: exceção Java do Jackson (`getCart` com `cartId="abc"`), NPE do Kotlin com nome de

classe interna, chave de tradução `item.quantity.invalid` (`updateCartItem` com `quantity=-2`). `retryable` só em User e Tracking; em User aparece `retryable: true` para erro de entrada do usuário. `getCart` de carrinho inexistente devolve `GEN-123 "try again"`, sem `retryable`. `ORDER_NOT_FOUND` vem com `retryable: false`, como deveria. O gateway responde HTTP 200 mesmo em erro.
- **Idioma:** o campo `suggestion` de um erro de Tracking ("Do not attempt to look up this Order ID again.") chegou à tela do cliente em inglês antes de cortarmos esse campo (17/10 ~21h50) [testado].
- **Impacto:** o agente decide entre tentar de novo, perguntar ou desistir lendo texto. Nosso `_erro_em()` reconhece os formatos um a um.
- **Sugestão:** envelope único `{error: {code,message,retryable,retryAfterSeconds,userMessage}, agentHints}`, sempre JSON, separando `retryable` de `needsUserInput`.

D10. Respostas que enganam [testado]

- Quatro situações diferentes viram "O item não está mais disponível": ID de linha antigo, `itemId` no lugar de `cartItemId`, item de outra loja, item realmente indisponível.
- `setVoucher` com cupom não aplicável responde sucesso; só um `agentHints` diz que não se aplica. `listVouchers` marca `available: true` para ele.
- `setCartDelivery` com horário inválido ou de outro método responde sucesso e troca o horário em silêncio (de amanhã 9h para hoje 20h).
- **Sugestão:** `CART_MERCHANT_MISMATCH`, `VOUCHER_NOT_APPLICABLE`, `TIMESLOT_NOT_AVAILABLE`, `INVALID_QUANTITY` como erro, não como sucesso.

D11. Preços e totais que não fecham [testado]

- `promotionalUnitPrice` maior que `unitPrice` em itens de `getMerchant` (70 → 115,50); o mesmo item com promocional diferente em `getMerchant` (115,50) e `getItem` (45,50); a segunda unidade sai a preço cheio sem aviso de que a promoção vale uma unidade.
- Carrinho de mercado: subtotal 7,39 + entrega 0 + taxa 1,99 = 9,38, mas `totalAmount` 5,54, sem linha de desconto nem nome de cupom. A diferença (3,84) é 52% do subtotal, e a conta de teste tem um cupom "52% OFF em mercados". Hipótese forte: cupom aplicado automaticamente sem a API dizer qual. No site, esse cupom só aplica depois de escolher forma de pagamento; na API o desconto apareceu sem pagamento definido.
- A conta de teste tem 193 cupons no site, e `listVouchers` devolveu 7 para o carrinho, sem explicar o filtro.
- `benefitsTotalAmount` sem relação estável com o total (às vezes maior que ele).
- `orderMinimumPrice` do carrinho veio 0 numa pizzaria em que `getMerchant` dizia 25.
- Pedido abaixo do mínimo: o site avisa ("O pedido mínimo para essa loja é de R$ 40,00") e desabilita o pagamento; a API deixou criar o carrinho (R$ 7,39, mínimo R$ 60), definir a entrega e seguir sem aviso.
- **18/10 ~01h58, bot no MCP real:** item de R$ 36,99 e total do carrinho R$ 47,97. Os R$ 10,98 eram entrega e taxa, que o MCP real já soma no total desde a criação do carrinho (o mock não somava). O bot mostrava como "seu pedido até agora" e parecia cobrança a mais; corrigido às 06h14 com uma linha "Com entrega e taxas".

- **Sugestão:** fonte única de preço; `discounts[]` com nome e valor de cada cupom (inclusive automáticos); `fees[]` com nome e valor de cada taxa; documentar `benefitsTotalAmount`; erro ou `agentHints` de pedido mínimo já no carrinho.

D12. CPF exigido só ao adicionar o item (MERCH-007) [testado]

- **Doc:** o CPF vai no `submitOrder` (`document` opcional) [só na doc].
- **Real:** loja que exige CPF responde `MERCH-007 "Este restaurante obriga a digitação do CPF"` já no `createCart` / `addItemToCart`. Vimos pela primeira vez em 17/10 ~19h e de novo em 18/10 no Nissei Mercado. O 06 original registra a mensagem com "restaurante" vinda de uma loja de farmácia/mercado.
- **Impacto:** o agente só descobre a exigência depois que a pessoa escolheu o produto. Antes da correção, o bot mandava "Tentar de novo" e a pessoa batia 3 vezes no mesmo erro (17/10); depois, em 18/10, o item se perdia e o CPF era pedido duas vezes.
- **O que fizemos:** 19h13 de 17/10, o erro passou a pedir o CPF sem a opção "sem CPF"; 08h16 de 18/10, o bot guarda a ação, refaz o item quando o CPF chega e não pede de novo; 09h01, se a loja recusa mesmo com CPF, a lista inteira vai para outra loja.
- **Sugestão:** `requiresCpf` em `searchOffers` / `getMerchant`, ou aceitar o CPF só no `submitOrder`; mensagem sem "restaurante" fixo.

D13. Endereço falso aceito e geocoding sem viés [testado]

- **Doc:** `createAddress` devolve `PLACE_NOT_FOUND` quando não acha [só na doc].
- **Real:** endereço inexistente devolveu um endereço na Califórnia com `matchConfidence: "SINGLE"`; "Rua das Flores, 10" devolveu 5 cidades sem viés pela localização do usuário; um CEP solto seguido do número deu um endereço no Canadá.
- **Impacto:** risco de entrega em endereço errado com confiança alta.
- **O que fizemos:** CEP é resolvido antes (BrasilAPI, com ViaCEP de reserva) e o iFood recebe "Rua, número - Bairro, Cidade - UF, CEP". Às 18h02 de 17/10 corrigimos também a leitura dos candidatos, que vinham dentro de `data` (segundo a mensagem do commit `4badde0`).
- **Sugestão:** restringir ao Brasil, aceitar a localização do usuário como dica e devolver `PLACE_NOT_FOUND` quando número ou cidade não batem.

D14. Descoberta pouco adequada a um agente de chat [testado]

- `getItem` devolveu 61 KB (~15 mil tokens) para um item (grupos de 12, 77 e 77 opções).
- `facets` de `browseOffers` calculados só sobre a página; `listItemTaxonomy` não existe no servidor (doc diz 24 tools, o servidor tem 23).
- `nextPageToken` solto da query, e a paginação por offset repete ou pula lojas porque o ranking muda entre chamadas.
- Query de 5000 caracteres volta ecoada duas vezes na resposta.
- **Relevância:** "cebola" trazia salgadinho (17/10, antes das 19h06); "pizza" trouxe "Omelete de pizza" de uma padaria (18/10 ~07h44) e "churrasco" trouxe carvão, sal grosso e carne moída como principal (horário não registrado); "carne friboi" num mercado real não trouxe nenhum produto da marca.

- `searchOffers` devolve até 25 itens por página; mostramos até 10 por vez e guardamos o resto para "Mais opções".
- **O que fizemos:** filtro de relevância por agente (Duda), receitas no código para o kit ("Omelete de pizza não é pizza") e `onlyOpen` desligável para testes de madrugada.
- **Sugestão:** paginar ou resumir `optionGroups`; limitar a query; facets sobre o universo; token preso à query; `brand` como campo filtrável.

D15. Lacunas de funcionalidade

- **Sem tool para listar pedidos** [testado]: `RECENT_ORDERS` veio vazio na conta de teste. O histórico só existe pelo `orderId` do `submitOrder`; guardamos o nosso.
- **Sem recorrência nativa** [só na doc]; construímos com `createCart(orderId)` e um modelo de pedido.
- **Cartão:** `setCartPayment` exige `cardTokenId`. `listPaymentMethods` devolve PIX e um cartão de teste com `cardTokenId` preenchido, mas não há API de tokenização [testado]. Por isso só PIX e vales. Não existe "pagar na entrega".
- **`submitOrder` devolve só `orderId`:** nenhum código PIX copia-e-cola ou QR em tool nenhuma [só na doc: visto no schema; a resposta do pedido real não foi registrada].
- **Sem cancelamento e reembolso** [só na doc]. **Sem push:** rastreio só por polling [só na doc].
- **Carrinho:** `createCart` já exige o 1º item (não existe carrinho vazio) [testado]; `addItemToCart` sem `options` estoura exceção, `options=[]` funciona e `createCart` aceita sem (a doc está certa e o schema errado) [testado].
- **Outros:** `paymentMethodId` é UUID (os exemplos dizem "PIX") [testado]; `sort: RATING` citado na descrição não existe no enum [testado]; `getUser` só tem `includeProfileData`, sem efeito visível [testado]; Discovery só aceita lat/lng, nunca `addressId` [testado]; timeslots existem em mercado (`timeslotId` na resposta, `selectedTimeSlotId` na entrada) e vêm omitidos em restaurante [testado]; a doc de User cita update/delete de endereço que não existem [só na doc]; o "fluxo completo de pedido" da doc pula passos que o Cart exige [só na doc]; o llms.txt recomenda Pending Action, que contas de teste com telefone falso não conseguem testar, e aponta um canal de suporte interno (Slack) que participante externo não acessa [só na doc]; a doc exige HTTPS no redirect mas o portal aceita localhost sem HTTPS [só na doc].

D16. Rastreio de um pedido real com status que assusta [testado]

- **Real (18/10, ~06h10–06h18, MCP real):** um pedido do Extra recebeu `PIX_CREATED` (06h10), `CONFIRMED` (06h11), `DISPATCHED` (06h14) e, 4 min depois, `PAYMENT_DECLINED` (06h18). O bot avisou "❌ O pagamento não passou" a quem já esperava a entrega. Se a conta era de teste e o que aconteceu com o pedido depois: não registrado.
- **O que fizemos (06h24):** `ORDER_CREATED` e `PIX_CREATED` ficam calados; recusa de pagamento depois de a loja confirmar fica só no log (se o pedido cair, o `CANCELLED` avisa); status que vai e volta avisa uma vez.

6.4 Doc × schema × realidade

Tema	Doc / schema diz	Realidade
Número de tools	24	23 (falta `listItemTaxonomy`) [testado]
`options` em `addItemToCart`	doc: obrigatório; schema: opcional	sem `options` estoura exceção [testado]
`paymentMethodId`	exemplos "PIX", "CREDIT_CARD"	UUID; o tipo fica em `kind` [testado]
`sort`	descrição manda `sort:RATING`	`RATING` não existe no enum [testado]
`getUser`	níveis BASIC/ADVANCED/BEHAVIOR	só `includeProfileData`, sem efeito visível [testado]
Erros do Cart	`retryable` e detalhes	sem `retryable`, código genérico [testado]
Carrinho vazio	createCart → addItemToCart	`createCart` já exige o 1º item [testado]
CPF	no `submitOrder`	exigido já ao adicionar item em algumas lojas (MERCH-007) [testado]
Escopo `openid`	obrigatório (llms.txt)	não permitido para o nosso cliente [testado]
`mcp.orders.read`	exigido para rastrear	não aparece no portal; `trackOrder` funciona sem ele no card [testado]
Endereço inexistente	`PLACE_NOT_FOUND`	endereço nos EUA com `SINGLE` [testado]
Total do carrinho	soma de itens	já inclui entrega e taxas desde a criação [testado]
Rate limit	existe, sem valores	403 HTML do Akamai por IP, sem `Retry-After` [testado]
Listar pedidos	—	não existe tool; `RECENT_ORDERS` vazio na conta de teste [testado]

6.5 O que funcionou bem

- Fluxo de endereço em dois passos (`createAddress` com `candidateId` e coordenadas) [testado].
- `listPaymentMethods` devolve PIX e cartão de teste com UUID [testado].
- O total bateu em todas as respostas de carrinho do QA, exceto no caso do mercado (D11) [testado].
- `agentHints` ajudam quando corretos (ex.: o erro de `setCartDelivery` com método inválido) [testado]; o polling do `trackOrder` por `agentHints.retryMessage` até `pollingStop` funcionou no rastreio real e dispensou intervalo fixo [testado].
- Erros de Tracking estruturados e acionáveis (`INVALID_ORDER_ID`, `ORDER_NOT_FOUND`) [testado].
- `createCart(orderId)` para refazer pedido antigo serviu de base para a recorrência [só na doc: no repo só está registrado o uso no mock].
- Leitura idempotente, UTF-8 e emoji preservados, paginação por token estável quando a lista não muda [testado].

- Aprovação do agente em menos de 1h e OAuth padrão (OIDC do `id.ifood.com.br` com PKCE S256 e as duas formas de autenticação do cliente) [testado].

6.6 Sugestões priorizadas

Os itens 1 a 7 seguem a ordem do entregável 06 original; 8 e 9 vêm das seções 7 e 8 dele.

1. IDs de linha estáveis no carrinho (D8).
2. Envelope de erro único e sempre JSON, sem vazar exceção interna (D9).
3. Códigos de erro semânticos no lugar de "item indisponível" e de sucesso falso (D10).
4. Preço com fonte única, `discounts[]` e `fees[]` explícitos, mínimo avisado no carrinho (D11).
5. Respostas do tamanho de um chat: opções resumidas, query limitada, paginação presa à query, `brand` filtrável (D14).
6. Geocoding restrito ao Brasil, com a localização do usuário como dica (D13).
7. `listOrders` e `grantedScopes` (D15, D3).
8. `429` com `Retry-After` e limite documentado por cliente ou token, não por IP (D6).
9. `requiresCpf` na descoberta, ou CPF só no `submitOrder` (D12).

6.7 O que não testamos

- Pagamento com cartão; loja fechada com entrega agendada; `customizations`; voucher aplicado com sucesso de forma explícita.
- `submitOrder` ficou fora do QA de 17/10. Em 18/10 o bot fez ao menos um pedido no MCP real (D16), mas o repositório não registra quantos, em que conta, nem o resultado do pagamento além do status do rastreio.
- O formato de `RECENT_ORDERS` com pedidos (veio vazio).
- Escopos efetivos do token (sem tool de introspecção; inferimos pelo comportamento).
- Pending Action (contas de teste não recebem push).

Anexo A — Cronologia do desenvolvimento (17/10 12h30 a 18/10)

Fontes: `git log` (horário de Brasília, -0300), `docs/prompts/*.md` (horários aproximados, com "~"), `docs/contexto-evento.md`, `docs/estado.md` e comentários no código. Quando o diário de prompts e o git divergem, vale o git. Itens do MCP remetem à parte 06.

Sábado, 17/10

Hora	Quem	O que
12h30	evento	Credenciamento, abertura e início
13h20–13h52	Fabio	Regras de colaboração multiagente, hooks de autoria e de arquivos protegidos
14h40–15h03	—	Portal de cadastro do iFood fora do ar (D1)
14h43	Fabio	Doc do WhatsApp: número e app da Meta emprestados do projeto `handwrite_bot`
~15h10	Davi	Agente "HackFood Agent" criado no portal; aprovado antes das 15h58
15h13	Fabio	Resumo da doc do MCP no repo (portal v0.27.0)
15h52–15h53	Davi	Docs de contexto: evento, referências, estado, decisões
15h59	Fabio	Claim do bot (`bot-ifood-agent`) e stub do MCP falso
~16h05	Fabio	2º agente, "Compras pelo WhatsApp", com redirect no backend publicado
16h18	Fabio	Back em Python no Modal: webhook, login do iFood (escopo inicial `openid`), cliente MCP
16h27	Fabio	Pedido completo no mock, 65 testes
16h30	Davi	`invalid_client` no `/token` do 1º agente (D2)
16h34	Fabio	Gemini lite na frente: ~1 s em vez de 10 s
16h37	Fabio	Bot no ar em modo demo; o número já atende pelo deploy novo
16h55	time	Ideia fechada e decisões: "a IA entende, o código garante"; confirmação nunca da IA
17h00	Fabio	Só PIX e vales (cartão exige `cardTokenId`)
até 17h05	Fabio	`/token` do 2º agente aceita o cliente: o `invalid_client` era do secret errado
até 17h09	Fabio	`openid` recusado; escopos só `mcp.*` (D3)
17h32–17h37	Fabio	Login só pela web, sem o OneLink do app
17h54–18h06	Fabio	Endereço: CEP resolvido antes do iFood; leitura dos candidatos corrigida (D13)
18h17–18h20	Fabio	Bot diz em qual conta iFood a pessoa entrou; ofertas em carrossel
~18h30	—	1º bloqueio do Akamai: ~50 chamadas de teste em poucos minutos, IP barrado ~20 min (D6)
18h41	Fabio	Sonda do 403 (`scripts/sondar_403.py`); rastreio corrigido
18h48	Davi	Achados do QA real do MCP registrados no `contexto-evento.md` §8
18h53	Fabio	Freio (~3 chamadas/s) e disjuntor no 403
19h06	Fabio	IA traduz a busca e filtra ("cebola" não é salgadinho) (D14)
19h13	Fabio	MERCH-007: loja que exige CPF passa a pedir o CPF (D12)

Hora	Quem	O que
19h17	Fabio	Disjuntor coletivo e botão de pânico do modo demo
19h18–19h46	Fabio	Recorrência: caderninho próprio, oferta na 2ª compra, relógio de demo
20h10	Pedro	Rascunho do dashboard
20h20–20h22	Davi	Estado e decisões; correção: `listPaymentMethods` devolve cartão de teste com `cardTokenId`
~20h27	—	Gemini com 503 e reserva com 429: o plano B buscou a frase inteira no iFood; key nova e outro modelo na fila
20h56–21h09	Fabio	Diário de eventos do bot e dash ao vivo
~21h05	organização	Callback em localhost confere só o path; edição da callback bloqueada no portal
21h12–21h15	Davi	Endereço não prende quem diz "oi"; sessão ociosa volta ao menu
21h17	Fabio	Regra do diário de prompts; sugestão para fechar o pedido mínimo
21h21–21h27	Fabio	Dash com seletor dados reais ou demonstração
21h23–21h25	Breno	Mascote no perfil do dash
~21h25 / 21h37	Fabio	Mock ligado em produção porque o iFood real cai muito; mock consistente commitado às 21h37
21h30–22h01	Pedro	Dash no Linux e PDF com indicadores agregados
21h40–21h51	Fabio	Resposta por áudio (ElevenLabs), voz pt-BR
~21h50	Fabio	Instrução em inglês do MCP chegando à tela; cortada na origem (D9)
22h02	Fabio	"O que a pessoa fala manda, o botão só ajuda"
22h09–22h29	Davi	"Olá" e "cancelar" não viram complemento; "cancelar pedido" é comando
22h32	Fabio	OpenRouter como reserva quando o Gemini cai
22h41	Fabio	Foto vira pedido e kit em grupo
22h51	Fabio	"Quero esse e pode fechar" vai direto ao checkout
22h58 / 23h00	Fabio	Commit `bbfd811` e deploy com foto, kit, voz e fixes (335 testes)
23h14	Davi	Cutucada (então 3 min) e carrinho esquecido aos 5 min
23h30	Fabio	"Qual o código?" e "tá chegando?" respondidos
23h36	Fabio	Vitrine no "oi"

Domingo, 18/10

Hora	Quem	O que
00h05	Fabio	A lista vai para o mercado que tem tudo e sai mais em conta
~00h15	Fabio	Levantamento das tools: 24 na doc, 19 usadas
00h48	Fabio	Status do número na Graph API conferido (TIER_250, qualidade GREEN)
01h04–01h09	Davi	Dash: Uso do MCP e rastro dos agentes de IA
01h–05h30	Davi	QA manual no WhatsApp, quase todo em mock (`docs/qa-18-10.md`)
01h29	Davi	"Mudar" digitado no pagamento (B1)
~01h31	Fabio	Deploy
01h40–01h56	Fabio	Agentes com crachá, fila própria e interruptor; "Agentes" vira a 1ª tela
~01h45	Fabio	Mock desligado: volta o MCP real
~01h58	—	No MCP real, "até agora" de R$ 47,97 com item de R$ 36,99 (D11)
~02h00	Fabio	Mock religado
02h01–02h31	—	A Meta parou de entregar respostas (erro 131042, faturamento da conta); resolvido pelo Fabio. Não é do MCP
02h09–02h13	Davi	"Sim" com sugestão pergunta de novo (B2); aviso de botão velho não empilha (B4)
02h10–02h17	Fabio	Lojas fechadas aparecem nos testes (`onlyOpen` desligável)
~02h30	Fabio	Mock desligado de novo
~02h40	Fabio	2º bloqueio do Akamai; sonda de outro IP recebe 200; redeploy e mock religado (D6)
02h51	Fabio	`scripts/destravar_ifood.sh`: redeploy limpo e disjuntor zerado em ~20 s
03h09	Fabio	Palco do dash anima em tempo real
03h36–03h43	Fabio	Sofia e insights com IA no dash
04h45–05h02	Fabio	Fixes: pedido mínimo, complemento, "Finalizar" com aviso (D7), kit parte do kit da tela
05h22	Fabio	Claude Sonnet 5.5 na frente de todos os agentes, Gemini de reserva
05h43–05h57	Davi	Fixes do QA: lembrete sem falso "preço mudou", itens somados, pagamento no lembrete, "hoje não", sugestões, foto sem relação
05h51	Fabio	Cutucada passa para 1 min
06h00	Davi	`docs/qa-18-10.md`
~06h10–06h18	—	Pedido real rastreado: `PAYMENT_DECLINED` depois de `DISPATCHED` (D16)
06h14	Fabio	M2 explicado (entrega e taxas no total); áudio fecha o pedido; "mais 10 leites" vira compra
06h20	Fabio	Modo palco e catálogo de reserva; kit respeita o orçamento

Hora	Quem	O que
06h24	Fabio	Rastreio não assusta com pagamento recusado depois da saída
06h31–06h41	Fabio	Pendências do QA (B3, U1, U3, U7, U9, U10, U17, U21, U24, H4, M3)
07h29	Fabio	Disjuntor local, aposentadoria do container e ponte por outro IP (8 containers, 5 IPs)
~07h44	—	Kit de pizza saiu com "Omelete de pizza" de uma padaria (D14)
08h11	Fabio	Quantidade na unidade de venda; botão "Revisar pedido"
08h16	Davi	MERCH-007 em loja real: item não se perde e CPF não é pedido duas vezes (D12)
08h19	Davi	Interruptor `cfg:cutucar` para a apresentação
08h20	Fabio	Troca de iFood sem a pessoa perceber
08h23	Fabio	"Já pode passar no pix" fecha direto; sugestão do resumo vale uma mensagem
~08h30	Fabio	Medição de latência pelo diário (D7)
08h31	Fabio	Beto consistente: receitas e ajuste do kit pelo código
08h31–08h40	Davi	Entregáveis 01 a 06 no repo
08h44	Fabio	Vina e Duda com Claude Haiku 5.5
09h01	Fabio	Loja que recusa mesmo com CPF leva a lista para outra loja
09h04–09h24	Fabio	Kit: "adiciona tomate" põe tomate; "12 ovos" é uma dúzia; "com 13 amigos" são 13
09h28 / 09h38	Fabio	Beto no Fable 5.1 e, 10 min depois, no Opus 5.5 (~8 s por kit)
10h26	Fabio	O projeto passa a se chamar AiFood
12h30	evento	Fim do desenvolvimento e submissão (agenda)

Lacunas da cronologia: antes das 18h40 (Davi) e das ~20h (Fabio) os prompts não foram registrados; os horários de deploy do Modal só existem quando alguém anotou; não há registro de quando cada pessoa parou de trabalhar.

Anexo B — Limitações conhecidas e o que não foi testado

B.1 Limitações do produto

- **Pagamento:** só PIX e vales; sem cartão nem "pagar na entrega". O `submitOrder` não devolve código PIX nem QR (D15).
- **Entrega:** só imediata; retirada e agendamento fora. Sem cancelamento ou reembolso pela API.

- **Uma loja por pedido;** lista sem loja que tenha tudo vai para a que tem mais, dizendo o que ficou de fora.
- **Kit em grupo:** foi o ponto mais frágil no QA; o ajuste agora é do código, mas o que o código não entende ("2 são vegetarianos") ainda vai ao modelo.
- **Quantidade** segue a unidade de venda lida do nome e da descrição; item sem unidade escrita pode ser contado errado.
- **MCP real instável em demonstração** por causa do bloqueio por IP (D6). O modo da demo (mock ou real) estava em aberto no último registro (H3).
- **WhatsApp:** mensagem iniciada por nós (template) bloqueada desde 17/10 (erro 141006); limite TIER_250; sem Groups API. O lembrete fora da janela de 24h espera a pessoa voltar.
- **Dash** roda só local (127.0.0.1); não está hospedado.
- **Botão de pânico:** o script `scripts/destravar_ifood.sh` está no repo; a skill que o acompanha ficou só na máquina do Fabio.

B.2 Não testado

- Com pessoas do público-alvo (nenhum teste).
- Cartão, loja fechada com agendamento, `customizations`, cupom aplicado com sucesso explícito.
- Quantos pedidos reais foram feitos e em que conta (só o rastreio de um está registrado).
- Escopos efetivos do token e Pending Action.
- Reteste completo das correções de 18/10 em produção depois do último deploy (o QA pedia reteste; resultado não registrado).
- Retorno do template da Meta (141006), não retestado para não enviar mensagem.